ODD protocol: an agentic agent-based model of nuraghe construction

Model. Nuraghe-construction agent-based model (ABM) by the authors, written in Python on repast4py (Collier and Ozik, 2022) with MPI, numba, numpy, pandas and scipy.

Version described. The code and configuration of the full audited run of 2 November 2026, with the v13 social switches and the v13g reputation rule (7.15).

- Code: `code/` (the v13 code: the frozen code of the run plus three opt-in social switches in `alliance.py`, off by default, and the v13g `reputation_rule` key, default `all`; with them off the outputs are identical to the frozen code, see 7.15 and `CODE_PROVENANCE.md`). Checksums: `CODE_SHA256SUMS` (frozen code, used by every run of the full audited queue) and `CODE_V13_SHA256SUMS` (v13). The repository copy differs from these only in comments and docstrings (`CODE_PROVENANCE.md`), so the line numbers below hold. Line numbers of `alliance.py` are those of the v13 file.
- Configuration: `config/config_full_audited.yaml`, cited as `cfg:<line>`.
- Cached derived inputs: `cache_dir` of the configuration (`cache/` below; rebuild instructions in `data/derived/README.md`).

Scenario described. `MOST_REALISTIC_AUDITED_LIT_P10` (run as `model.py <config> --alliance MOST_REALISTIC_AUDITED_LIT_P10 --seeds 1-100`). `alliance.scenario` (`code/alliance.py:1958-1998`) resolves it to the `MOST_REALISTIC` rule set of `alliance_extra` (cfg:392-394) with the audited work-party rule (cfg:624) and phasing in 10% cohorts (name suffix `_P10`, `code/phasing.py:38-45`). With this configuration the resolved scenario is identical, key for key, to `MOST_REALISTIC_P10`.

How rules are tagged. Every rule and number below was read from the code or the configuration. Code locations are given as `file:function (lines)`. Variable and function names are those of the code, as Grimm et al. (2020) recommend for linking an ODD to its implementation. Where an older project document disagrees with the code, the code is described and the disagreement is listed in Appendix D.

Guidance followed and version of the protocol

This description follows the ODD protocol as updated by Grimm et al. (2020), with the current practice reviewed by Grimm et al. (2027). It uses the seven numbered elements and the eleven design concepts of Grimm et al. (2020), as tabulated in Grimm et al. (2027, Table 1). The points of the 2027 review applied here:

1. **Seven numbered elements as section titles.** "Overview", "Design concepts" and "Details" are not used as section titles (Grimm et al., 2027, Sect. 6.1).
2. **Purpose and patterns.** The overall purpose is stated with the purpose categories of Edmonds et al. (2019). It is kept separate from the specific research questions and from the patterns used to judge the model (Grimm et al., 2027, Sect. 2).
3. **Input data in the strict sense.** "Input data" means external drivers that change over simulated time (Grimm et al., 2027, Sects. 2, 6.1). This model has none. The spatial datasets are listed in Section 6 for credit and reproducibility, labelled as initialization data.
4. **Process overview names processes only.** Their behaviour is in Section 7 (Submodels), not in Section 3 (Grimm et al., 2027, Sect. 6.1).
5. **Complete parameter information.** Every parameter acting on the headline run is given with its definition, unit, value, published range where one exists, source and evidence status (Appendix A; Grimm et al., 2027, Sect. 6.1).
6. **Links to code.** Each submodel is tagged with the functions and lines that implement it (Grimm et al., 2027, Sect. 6.3).
7. **Extension for human decision-making: ODD+D.** The communities decide whether to ask for help, whether to accept a request, which quarry face to use, how far to relay stone and whether to imitate a neighbour. The 2027 review names ODD+D (Müller et al., 2013) as the extension for decision submodels of human agents (Sect. 4, Table 2) and names no newer variant for that purpose. We therefore add the ODD+D elements to the 2020 structure, which is otherwise left unchanged:
   - Theoretical and empirical background (4.0), Individual decision-making (4.12) and Heterogeneity (4.13), as extra design concepts;
   - the three extra Overview questions of ODD+D: for whom the model is designed (1.4), its exogenous drivers and how space is included (2.7);
   - Implementation details (7.0), as an extra element of the details.

ODD+2D (Laatabi et al., 2018, in Grimm et al., 2027, Table 2) documents the data behind design and parameterisation. It was not adopted as a structure. Its content is covered by Section 6 and the parameter audit (Appendix A, Appendix C).

8. **Rationale layer.** Each submodel ends with a short Rationale. The full justification of every value is the project's parameter audit and traceability record, mapped onto the elements of TRACE (Schmolke et al., 2010; Grimm et al., 2014; Ayllón et al., 2021) in Appendix C. Grimm et al. (2027, Sect. 5.2) recommend TRACE documents for recording the whole modelling process. Parameterisation and validation are not part of an ODD itself (Grimm et al., 2027, Sect. 7.4), so they are kept in the appendices.
9. **Summaries and variants.** A text version of the visual-ODD panels (Szangolies et al., 2024) comes next. The scenario families are given as "delta" descriptions against the headline (Grimm et al., 2027, Sect. 6.2; Appendix B).

10. **Edge cases and the use of AI.** The 2027 review warns that descriptions drafted with large language models can look complete while leaving out edge-case behaviour, and asks for a human-in-the-loop process (Sect. 7.2). This description was drafted by the modelling agent (an AI research agent) from the frozen code and configuration. Every statement is tagged to the code, and the edge cases and inactive branches are listed in 7.13. An independent AI agent then verified the description against the code and configuration (161 claims, 8 errors); the errors are corrected in this version. The v13 social mechanisms and the v13g reputation rule (7.15 and the lines that refer to it) were added later, checked sentence by sentence against the v13 code by the modelling agent, and were not part of that independent verification. Human review by the PI was still pending when this was written.

Summary panels (text version of a visual ODD)

Panel	Content
Purpose	A tool for testing archaeological ideas about how nuraghi were built (labour, time, cooperation). Sardinia, with 7,167 nuraghe sites, is the test case.
Entities	7,167 community agents (one per nuraghe), their towers as chains of stage tasks, pairwise trust, quarry faces and routes on a 30 m grid.
Initialization	Per seed: residents and work party per site; block masses; pieces per tower; stage labour; cohort membership. Static: least-cost routes, technique rules, sea faces.
Submodels	Tower start (choice of face, help and relay; imitation), help requests (acceptance and trust), joint work (join mode), daily work (upstream cap), skill and learning, tower completion (value update), phasing (10% cohorts).
Scenarios	The headline plus one-switch families (Appendix B).
Observation	Per site and seed: man-hours by stage, own, received and given, start and finish, work party, tonnes. Per pair: trust, help, asks. Island man-hours and island years per seed.

1. Purpose and patterns

1.1 Overall purpose

The model is a tool for testing archaeological ideas about how nuraghi were built: who quarried, dressed, hauled and lifted the stone, with how much labour and time, and with what cooperation between communities. Each idea becomes an explicit, seeded simulation rule, and its consequences are computed for every nuraghe of the island. Sardinia is the test case; the method is the subject. The paper draft frames the aim as using the link between agentic AI and ABM proposed by Anonymous (2026) to explore the literature on how nuraghi could have been built, and to show the benefit of that link for exploring the range of ways in which nuraghi may have been built and how long building took across Sardinia. In the purpose categories of Edmonds et al. (2019), the model serves **theoretical exposition**: it works out what stated, literature-based assumptions imply. It is not built for prediction. Its results bound what the assumptions allow; they do not forecast what happened.

1.2 Specific questions

The paper draft states three questions:

1. What does a construction model that combines the best-supported evidence imply about the labour, time and cooperation needed to build the nuraghi?
2. Which assumptions matter most for those answers, and where are the limits of feasibility?
3. Why is modelling construction in this way useful for Nuragic archaeology?

The headline scenario answers question 1. The scenario families (Appendix B) and the parameter alternatives address question 2.

1.3 Patterns used to evaluate the model

The headline parameters were **not calibrated** to any pattern: every value comes from the literature or is a labelled assumption (Appendix A). The patterns below are the ones the code actually checks or reports. No pattern on the spacing of nuraghi is used. The closest spatial check is the density-against-stone test (g). **Patterns b-i are pending re-run on the version described** (see below).

#	Pattern (reference)	Where it is checked	Use
a	Single tower about 3,600 man-days (Webster, 1991, p. 852, Table 1)	`code/tests/verify_webster.py` (accepts 0.5-1.5 times 3,600 man-days and 100-1,100 transport man-days for a single tower about 200 m from its outcrop, `verify.webster_target_haul_m` cfg:104); `code/tests/verify_alliance.py` (A0_verify reproduces the `webster` scenario)	Verification of the baseline labour model that the alliance model inherits

#	Pattern (reference)	Where it is checked	Use
b	Local stone: 7 of 113 Marghine nuraghi used stone from more than 2 km (Gallin 1989 via Webster 1991: 848, REPORTED-ONLY)	`code/tools/v12_checks.py` check V_gallin (share of sites in 9 Marghine comuni with a route over 2 km, against a Wilson 95% interval)	Pattern for the least-cost stone rule
c	Build span of a single tower about 3 years at 40 days a year; Porcarzos additions 4-5 years (Webster 1991: 853); 12 and 25-30 years (attributed to Webster by Atzeni & Frau 2023: 326, REPORTED-ONLY)	V_duration	Pattern for site build times
d	About 9.5 days a year per adult man, 2.5% of adult male time (Webster 1991: 853)	V_labour_share	Pattern for the labour burden
e	About 208 sites and 4,160 people under construction at once (Atzeni & Frau 2023: 326-327)	V_concurrency	Pattern for simultaneity
f	Island span about 500 years (Ialongo 2018) or 700 years (Ugas via Moravetti et al. 2009: 361, REPORTED-ONLY)	V_span	Comparison of island years (labour-limited time) with chronology
g	Nuraghe density rises with stoniness (Araque Gonzalez 2021: 194-195, opinion)	V_density_stone (Spearman and partial Spearman on about 5 km cells, using the outcrop raster and the soil map)	Test of an idea on the inputs, not on model output
h	Unfinished nuraghi Mura Era I and Portazo (Atzeni, A. 2019: 92)	V_unfinished	Pattern for the slowest sites
i	Route lengths at named sites (published provenance; Sarcidano; San Gavino)	V_provenance, V_sarcidano, V_sangavino	Pattern for routes
j	Most a single community can supply: about 30,000 person-hours (Wunderlich et al., 2024)	`code/report_alliance.py:write_scenario` (`pct_sites_over_30k`, threshold cfg:194)	Reported share of nuraghi above the threshold

The island population of at most 300,000 (Calò et al., 2008, p. 41) is used as a **constraint** (7.3), not as a pattern. The checks b-i in `code/tools/v12_checks.py` read the outputs of the earlier v11 run (paths written into the script). They have not yet been re-run on the outputs of the full audited run (Appendix D, item D17). **Status on this version: the checks b-i are pending re-run on this version**; until then their results are those of the v11 run and are not evidence about the version described here.

1.4 For whom the model is designed (ODD+D)

- **Archaeologists** working on Nuragic Sardinia and on the energetics of monumental building, who want to see what a stated assumption about labour, transport, cooperation or phasing implies when it is applied to every nuraghe of the island.
- **Modellers of past societies**, as a worked example of an agent-based model built, run and documented by an agentic AI under the direction of the principal investigator (PI) (1.1; Appendix C).
- It is a research tool for theoretical exposition. It is not designed for stakeholders, management or policy decisions. Users are expected to change the scenario switches of the configuration (Appendix B) and to read results as bounds implied by the assumptions, not as forecasts.

2. Entities, state variables, and scales

The headline runs the alliance model (`code/alliance.py`). The `Builder` and `Site` agents of `code/model.py` are used only by the baseline, cooperation and literature families; they take no part in the headline.

2.1 Community agents

There is one `Community` per nuraghe site (`code/alliance.py:229-234`), a repast4py `core.Agent` held in a `SharedContext` (`AllianceModel.__init__`, lines 244, 558-561). The agent object carries only its id (`gid`). Its state lives in arrays of `AllianceModel` indexed by `gid`:

State variable (code name)	Meaning	Unit / type	Set / changed
`gid`, `UID`, `uid_int`	index, site identifier, numeric part of the UID (RNG key)	int / str	static
`TWR_N`	towers of the nuraghe (1-7, keep included)	int	static (input)
`x`, `y`	site position	m, EPSG:25832	static (input)
`residents[i]`	residents of the community	persons	initialization (7.3)
`B[i]`	work party (builders)	persons	initialization (7.3)

State variable (code name)	Meaning	Unit / type	Set / changed
`cap[i]`	labour capacity per working day = `B[i]` x `hours_per_day`	person-hours/day	initialization
`towers[i]`	per tower: (tonnes, crews at 100 kg/person, crews at 300 kg/person)	t, persons	initialization (7.4)
`tower_tech[i]`	per tower: (crews at the site's friction, crews at mu 0.38, crews at 100 kg/person, splitting man-hours)	persons, man-hours	initialization (7.4)
`tower_nblk[i]`	pieces per tower after splitting	count	initialization
`tower_sea[i]`	per tower, values for a sea face with the payload split (or None)	dict	initialization
`site_stage[i]`	per-tower labour of the quarry, dress and build (incl. lift) stages before learning and skill	person-hours	initialization (7.2)
`site_lift_share[i]`	lift share of the build stage (accounting only)	fraction	initialization
`S.tech["site"][i]`	the site's technique rule: quarry rule, dress rule, lift rule, friction mu, haul rule (short / long)	dict	static (7.2)
`S.tech["stone"][i]`	quarry stone class used for density and hardness	str	static
`tower_idx[i]`	towers finished	count	dynamic
`current[i]`	current tower: strategy, task ids, helpers per stage, join shares	dict / None	dynamic
`start_day`, `finish_day`, `start_season`, `finish_season`, `phase`	timing and cohort	day, season, int	dynamic
`Q[i][dim]`	value table per decision dimension (`DIMS`, line 71)	dict action -> value (negative man-hours per tonne)	dynamic (7.10)
`last_strategy[i]`	last strategy and its own man-hours per tonne	tuple	dynamic
`cum[i, s]`	experience in skill stage s (quarry, dress, build), in tower-equivalents	float	dynamic (7.9)
`rng[i]`	decision random stream	numpy Generator	initialization
`acc[k][i]`	man-hours: `mh_total`, `mh_own`, `mh_received`, `mh_given`, `mh_quarry`, `mh_dress`, `mh_transport`, `mh_build`, `mh_lift`, `mh_setup`, `mh_track`, `mh_walk`	person-hours	dynamic (7.12)
`defector[i]`	never helps (all False in the headline, `defector_frac` 0)	bool	initialization

2.2 Pair state (relationships)

Variable	Meaning
`trust[(a, b)]`	a's trust in b, in [0, 1]; default `trust0` (T, line 1110)
`received_from`	set of (a, b): a has received help from b
`allied_season[(a, b)]`	first season with mutual trust >= `alliance_threshold` (recorded only)
`help_mh`, `help_stage`, `asks`, `asks_stage`, `help_log`	bookkeeping of help given and of asks and refusals, per pair, stage and season
`soc["log"][j]`	(v13 `reputation` only) j's decisions as an asked community, in time order: (requester, good). A help is good, an unjustified refusal bad; justified refusals are not logged. Under `reputation_rule: gentle` only a defector's refusal or a refusal after help received from the requester is logged as bad; other refusals are not logged (7.15)
`soc["cheated"]`	(v13 `partner_choice` only) set of (a, b): b refused a, without justification, after having received help from a, and has not helped a since
`soc["adj"][i]`	(v13; used by `tie_imitation`) communities linked to i by help given or received with mh > 0

The `soc` state exists only when a v13 switch is on (`self.soc is None` otherwise; `AllianceModel.__init__`, lines 502-504).

2.3 Tasks

These are not agents. A tower is a chain of stage tasks held in growing arrays (`add_task`, lines 1129-1142):

- `t_perf`: the performing community;

- `t_total`, `t_done`: labour total and labour done, in person-hours;
- `t_up`, `t_up2`: the upstream task(s);
- `t_final`: whether this is the tower's last task;
- `t_finish`: the day it finished;
- `t_stage`: its index in STAGES = quarry, dress, track, leg1, transport, build (line 66).

2.4 Quarry faces and routes

These are static, per site (`Static.__init__`, lines 126-196). A site has up to `k_quarries` = 5 faces, each with:

- route index, outcrop region, quarry cell;
- `C`, the route's drag cost at mu 0.3 (flat-metre equivalents), and `L`, its length (m);
- the hand-off move for each relay fraction.

A sea face (route -2) is added when the site's 2-knot sea table gives the sea route (46 sites; `cache/sea/sites_haul_sea2kn.csv`). It carries the trip time `trip_h`, the land part `land_h`, the boat part `boat_h` and the hoist labour per block `hoist_ph`.

2.5 Environment and globals

- **Grid.** 30 m cells, 4,808 x 9,080 (W x H), EPSG:25832 (cfg:20); transform stored in `cache/alliance/kbest.npz`.
- **Static grid products:**
  - the 8-band directional drag cost raster (mu 0.3; cfg:15) and its mu 0.38 and mu 0.1 rebuilds (`cache/cost_mu/`);
  - the outcrop code raster and its regions;
  - per-move arrays along every face route: from-cell `mv_cell`; costs `mv_c03`, `mv_c01`, `mv_c038`; length `mv_len`.
- **Neighbour lists.** Communities within `R_m` = 10 km, straight line, sorted by distance, then id (lines 186-196).
- **Globals:**
  - `open_faces` (region -> communities that opened it);
  - `track_owner` (empty: tracks are off);
  - `handoff_use`;
  - `season`, `day`, `n_started`, `cur_phase`, `cohorts`.

2.6 Scales

- **Time step.** One repast4py tick is one building season of `season_days` = 90 working days (cfg:255), which stands for one year (`season_step`, lines 1753-1821; `schedule_repeating_event(1, 1, ...)`, line 562).
- **Within a tick.** A day loop of 8-hour working days (`hours_per_day`, cfg:58).
- **Run length.** The run ends when every tower is built, with a safety stop at `max_seasons` = 20,000 (cfg:393; line 563).
- **Space.** 30 m cells over Sardinia.
- **Extent.** The 7,167 nuraghe sites of the input.

2.7 Exogenous drivers and space (ODD+D)

- **Exogenous drivers.** None change during a run: there is no climatic, demographic, economic or chronological forcing. What drives a run is fixed at initialization: the site list and tower counts, the terrain (drag-cost rasters), the outcrops and recorded stone, the work-party rule and the cohort draw (Sections 5, 6). The population ceiling is a fixed constraint (7.3).
- **How space is included.** Explicitly, on a 30 m raster (EPSG:25832) over Sardinia:
  - haul labour follows least-cost routes over a slope-dependent directional drag-cost surface, precomputed per site and face (7.1) and costed move by move (7.5);
  - interaction is limited by straight-line distance: requests, relays and imitation within `R_m` = 10 km, acceptance falling by 0.2 logit per km, and helpers' walking charged by distance (7.7);
  - relay helpers must be nearer the quarry than the asker (7.7), and 46 coastal sites have a sea face with a boat route (7.1, item 6).

    - Communities do not move; only stone and work parties travel, and travel is charged as labour.
- **Resolutions and extents** are given in 2.6.

3. Process overview and scheduling

Each seed is one independent model run (`alliance.run`, lines 2048-2136). Each MPI rank runs whole-island models for its share of the seeds (`todo[rank::nwork]`, `nwork <= max_ranks` = 8, cfg:279) on `MPI.COMM_SELF` (line 238).

```
initialise (Section 5)
repeat each tick = season (AllianceModel.season_step):
    season += 1
    if no community has a current tower and a cohort remains:       # phasing (7.11)
        for each community of the next cohort, in gid order:
            start_tower: tower decisions (7.6) and task creation with help requests (7.7)
    days_left = season_days
    while days_left > 0:
        order active tasks by (stage index, task id)                 # rebuilt when tasks change
        if no active task: skip the rest of the season; break
        run_days: advance whole days; stop at the end of the first day
                  on which some tower's final task is complete (7.8)
        for each community whose current tower is complete, in gid order:
            finish_tower: value update (7.10); start the next tower at once (7.6, 7.7)
    if every community has started, none has a current tower and no cohort remains: stop
write per-seed output files (4.11, 7.12)
```

- **Order.** Starts and completions are processed in `gid` order. Within a day, each performer's hours go to its active tasks in global order of stage, then creation (7.8).
- **v13 social switches (off in the headline; 7.15).** When on, each request decision passes through the social gate (partner choice, then reputation) before the trust update; candidate lists are reordered by exchange history (partner choice); the imitation pool is the help-tie partners (tie imitation). Order and timing are otherwise unchanged; the updates are immediate, like trust.
- **Updates.** Trust is updated at once, at each request (asynchronous). Labour is charged to the accounts when a task is created (7.12).
- **Time.** Time is discrete in whole days. Events (tower completions) are handled at the end of the day on which they happen.

4. Design concepts

4.0 Theoretical and empirical background (ODD+D)

- **General concepts behind the design.** Architectural energetics (labour = quantity x task rates; Webster 1991; Boswinkel 2021), the chaîne opératoire as a chain of dependent tasks, least-cost movement of heavy loads over terrain (Boswinkel 2021: 43-44), and the archaeological discussion of cooperation between Nuragic communities (Webster 1991; Araque González 2021). These give the structure of the model (4.1).
- **Basis of the decision model.** The communities' decisions rest on general behavioural theory, not on data about Nuragic decisions, which do not exist:
    - bounded rationality with reinforcement learning of action values (a stateless epsilon-greedy bandit; Sutton & Barto 2018: 28, 32);
    - social learning by imitation of a more successful neighbour (Gilbert 2020, s. 1.3.6);
    - reciprocity: trust rises when help is given and falls when a community that was helped refuses (mechanism specified by the authors);
    - Wright's (1936) learning curve for skill.
- **Why this decision model.** The purpose is theoretical exposition (1.1). The decision rules are kept simple, explicit and switchable, so that the consequences of each idea can be isolated (Appendix B). A richer model of choice would add parameters that no evidence could constrain.
- **Empirical basis of the submodels.**
    - Labour and transport rates, stone quantities, block sizes, pulling force, speeds and the work-party rule come from published experimental, ethnographic and archaeological figures (Appendix A: status SOURCED, DERIVED or REPORTED-ONLY).
    - Site list, tower counts, recorded stone, lithology and elevation are empirical data (Section 6).
    - The parameters of the decision and trust rules (betas, trust steps, epsilon, alpha, imitation probability, radius, number of asks) have no empirical source and are labelled assumptions (Appendix A).

- **Level of the empirical data.** Site level (one nuraghe, its towers and stone), cell level (terrain, outcrops) and island level (the population ceiling). No individual-level data are used, and nothing is calibrated (1.3).

4.1 Basic principles

- **Architectural energetics.** Labour = stone quantity x task rates, stage by stage.
  - Quantities and rates: Webster's (1991, Table 1) man-days per tower for dressing and walling; Boswinkel (2021) for quarrying, facing and lifting.
  - Transport is simulated. Each block needs a full crew sized by the pulling force per person against the sledge friction.
  - Hauling follows least-cost routes over a slope-dependent drag-cost surface (7.1).
- **Chaîne opératoire as a task chain.** Quarry, then dress, then transport (optionally in two relay legs), then build and lift. No stage may run ahead of the stage before it. This generalises the delivered-stone cap of the baseline model (`code/alliance.py`, module docstring).
- **Social mechanisms:**
  - help between communities, asked for and granted with a probability that rises with trust (principle specified by the authors);
  - learning of strategies: a stateless epsilon-greedy update of action values (Sutton & Barto 2018: 28, 32);
  - social learning by imitation (Gilbert 2020, s. 1.3.6);
  - a Wright (1936) learning curve for skill;
  - construction phased in cohorts (specified by the authors).
- **Hypotheses.** The model does not choose between hypotheses about Nuragic social organisation; each one becomes a scenario switch (Appendix B).

4.2 Emergence

Emergent:

- island years and per-site build spans;
- island and per-site man-hours (through the choice of face and route, relays, splitting, learning and skill);
- the share of labour received as help;
- the help and request networks, alliances, hubs and specialisation;
- the strategy mix over time.

Imposed:

- the technique rule of each site;
- the stone mass;
- the per-tower stage labour before learning;
- the work parties;
- the cohort membership.

4.3 Adaptation

At the start of each tower a community chooses (`start_tower`, lines 1532-1546; `choose`, lines 1306-1346):

- its quarry face (one of up to 5, or a sea face);
- whether to ask for help with quarrying, dressing and building (each 0 or 1);
- the relay fraction (0, 0.25, 0.5 or 0.75).

The track dimension is absent because `tracks` is false. An asked community decides whether to accept (7.7). The choices are indirect objective-seeking: they follow learned values, imitation and random exploration.

4.4 Objectives

- **Tower choices.** The reward after a tower is r = -(the community's own man-hours on that tower / the tower's tonnes) (`finish_tower`, lines 1731-1739). Time does not enter the objective, nor do labour received and labour given.

- **Acceptance.** Not objective-driven: it is a logistic function of trust, backlog and distance (7.7).

4.5 Learning

- **Value learning.** One value table per decision dimension (7.10). After each tower: Q(a) <- Q(a) + alpha (r - Q(a)), alpha = 0.5. An action's first value is its first reward. Untried actions count as 0, which is optimistic because rewards are negative. The quarry-face values start from a model-based estimate (4.6).
- **Social learning.** Imitation, with probability 0.3 per tower (7.6).
- **Skill.** A Wright curve on experience in quarrying, dressing and building. With `specialisation` on, help given also counts as experience (7.9).
- **Within-site learning (S8).** Tower t (0-based) costs 0.85^t of the first tower in the craft stages; a task takes the larger of this discount and the skill discount (7.9).
- **Trust.** A memory of reciprocity, updated at every request (7.7).

4.6 Prediction

There is no explicit forecasting. The only anticipation is the prior of the quarry-face values on the first tower: the community's own estimate of man-hours per tonne for each face, as transport labour plus its stage labour plus the setup cost (lines 1547-1552). Communities do not predict the behaviour of others.

4.7 Sensing

The deciding community knows:

- the costs of its own faces;
- its own trust in others;
- the last strategy and cost per tonne of every community within 10 km (for imitation);
- the skill factor of its neighbours (requests go first to the most skilled);
- for relays, which neighbours are nearer the quarry than itself.

The asked community knows:

- its trust in the asker;
- its own backlog (remaining labour of its active tasks / its daily capacity);
- the straight-line distance.

Sensing has no cost and no error.

4.8 Interaction

Direct:

- requests and acceptances (up to 3 asks per task, 7.7);
- joint work on a stage (join mode: the helper adds hands to the owner's stage);
- relays (a helper joins the owner on the first leg of the transport).

Indirect:

- a helper's hours are drawn from its daily capacity, which delays its own tasks;
- an asked community's backlog lowers acceptance;
- imitation of neighbours.

Requests carry no labour cost in the headline (brokers exist only in the pre-network family). Helpers' walking is charged to the helped nuraghe.

4.9 Stochasticity

Random:

- residents per site (uniform within the range of the tower class);
- block masses (clipped lognormal);
- cohort membership (random permutation per seed);

- the decision draws: imitation, epsilon exploration, ties between equal values;
- the acceptance draws.

Deterministic:

- technique rules;
- routes and costs;
- stage rates.

The random streams are listed in 7.14. Seeds 1-100 (cfg:98).

4.10 Collectives

- **Alliances.** Pairs with mutual trust >= 0.7 (`allied`, lines 1113-1115). They are recorded but change nothing in the headline: alliances act only through shared trackways and shared quarry faces, and both are off.
- **Cohorts.** The phasing cohorts are collectives imposed by the schedule.
- **Households.** Not represented in the headline.

4.11 Observation

Per seed (`AllianceModel.results`, lines 1843-1940; `alliance.run`, lines 2093-2108), written to `cache/runs/alliance_<scenario>/<signature>/`:

File	Content
`seed_NNNN.sites.csv.gz`	per site: `seed`, `gid`, `TWR_N`, the 12 man-hour accounts (`mh_total` ... `mh_walk`), `builders`, `defector`, `start_season`, `finish_season`, `years` = finish_season - start_season + 1, `days` = finish_day - start_day + 1, `tonnes`, `phase`, `start_day`, `finish_day`
`seed_NNNN.pairs.csv.gz`	per pair: trust both ways, help both ways (total and per stage, with task counts), asks and refusals both ways (total and per stage), `allied`, `ask_only`
`seed_NNNN.strategy.csv.gz`	per tower started: season, quarry face, the three ask choices, relay fraction, track, imitated
`seed_NNNN.spec.csv.gz`	per site: labour given per stage, skill factor and experience per skill stage
`seed_NNNN.handoffs.csv.gz`, `.tracks.csv.gz`	relay hand-off cells and their uses; track cells (empty in the headline)
`seed_NNNN.helplog.csv.gz`, `.allied.csv.gz`	help man-hours per season, helper and owner; first allied season per pair
`seed_NNNN.population.json`	residents, active peak, ceiling, whether the ceiling binds, redraws, scale
`seed_NNNN.done`	number of seasons simulated

Per scenario (`code/report_alliance.py:write_scenario`, lines 50-193), in `outputs/scenarios_alliance/<scenario>/`:

- `nuraghe_manhours_runs.csv.gz` (all site-runs);
- `nuraghe_manhours_summary.csv` (per site over seeds: mean, min, max and sd of `mh_total`; mean own, received and given; days mean, min and max; mean years and builders; mean man-hours per stage);
- `strategy_by_season.csv` (25-season bins);
- `specialisation.csv`;
- `stats.json`:
  - island man-hours, mean, min and max over seeds;
  - stage shares and the share received;
  - median man-hours of 1-, 2- and 3-tower sites;
  - `pct_sites_over_30k`;
  - island seasons, mean, min and max of the maximum `finish_season`;
  - median years of single towers;
  - alliances per seed, mean and largest alliance size;
  - share of towers with a relay.

- Shapefiles (`nuraghe_manhours.shp`, `alliances.shp`, `handoffs.shp`), except with `--no-shapefile`, which the full chain used.

Summary tools (`code/tools/v12_summary.py`, `code/tools/family_summary.py`), per seed:

- island man-hours `island_MH` = sum of `mh_total` over the 7,167 sites;
- island years = (max `finish_day` + 1) / 90;
- `site_years_mean` = mean of days / 90;
- `transport_share`, total `tonnes`, total `builders`.

Against a reference run, paired by seed: the percent change, with a 95% t interval over seeds. The verdict "moves results" requires the interval to exclude 0 and the change of the means to be at least 1%. The code thus has two island-time measures: day-based years (summary tools) and the count of seasons (`stats.json`).

4.12 Individual decision-making (ODD+D)

Question	In this model
Subjects and objects of decisions	The community (the whole work party acts as one) decides on the quarry face, asking for each craft stage, and the relay fraction; asked communities decide whether to accept
Level	Community only; no decisions by individual builders
Decision model and rationality	Bounded: epsilon-greedy choice on values learned from the community's own experienced cost; imitation of a cheaper neighbour; no optimisation over the others' responses
Adaptation to changing conditions	Through value updates after each tower and through trust; acceptance reacts at once to the asked community's backlog
Social norms	Reciprocity: a refusal after having received help from the asker lowers the asker's trust in the refuser by 0.2; accepted help raises it by 0.1
Spatial aspects	Requests, relays and imitation within 10 km straight line; relay helpers must be nearer the quarry than the asker; acceptance falls by 0.2 logit per km
Temporal aspects	Decisions are made once per tower, at its start; there is no re-planning during a tower; values carry over between towers
Uncertainty	No explicit representation; uncertainty is handled only through exploration (epsilon = 0.1)

4.13 Heterogeneity (ODD+D)

- **What differs between communities:** tower count, location, stone, routes and faces, technique rule (quarry, dress, lift, friction), work party, and cohort.
- **What is shared:** the decision rules and their parameters.
- **What diverges:** learned values, trust and skill.

5. Initialization

Initialization runs once per process (static data) and once per seed (`AllianceModel.__init__`, lines 238-563).

5.1 Static data, the same for every seed

`Static.__init__` (lines 126-196) and `Static._init_tech` (lines 199-225) set up:

- the site table `cache/sites_haul.csv`;
- the k-best faces from `cache/alliance/kbest.npz`. Of the 7,167 sites, 7,165 have 5 faces and 2 have 1; there are 35,827 routes;
- the technique rules from the mu 0.38 and mu 0.1 haul tables (7.2) and the mu 0.38 move costs (`cache/alliance/kbest_mu038.npz`);
- the sea faces from `cache/sea/sites_haul_sea2kn.csv`;
- the neighbour lists.

5.2 Per seed, for each site i in gid order, from its stream `default_rng([seed, uid_int])`

1. **Residents.** The ceiling pre-pass draws them with the site's first uniform (7.3). The builders follow from the residents. A dummy uniform is then drawn so that the block draws are the same as without the ceiling (lines 327-329).
2. **Blocks**, then their assignment to towers (7.4).

3. **Pieces, crews and splitting man-hours** for each tower; sea payload pieces if the site has a sea face (7.4).
4. **Stage labour per tower** (7.2), learning off.

Resulting work parties: builders = max(1, floor(0.4 x residents + 0.5)) with residents ~ U[lo, hi]:

Towers	Residents (cfg:384)	Builders (range)
1	5-40	2-16
2	40-110	16-44
3	110-185	44-74
4	185-260	74-104
5	260-330	104-132
6, 7	330-400	132-160

5.3 Decision and schedule state

- **Value tables:** empty; the quarry-face prior is filled when the first tower starts.
- **Trust:** default 0.5. Experience: 0. Last strategies: none.
- **Decision streams:** `default_rng([seed, uid_int, 7])`.
- **Run stream** `default_rng([seed, 99991])`: draws the start order, which is unused under P10. No defectors are drawn.
- **Cohorts:** `phasing.cohorts` (7.11).
- **Clock:** season 0, day 0. No community has a tower until the first tick.

The initial state differs between seeds only through the residents, the blocks, the cohorts and the decision streams.

6. Input data

In the ODD sense, the model uses no input data: no external driver changes during a run. All datasets below are read before or at initialization, most of them through preprocessed products in `cache/`. They are listed here for credit and reproducibility. All layers are in EPSG:25832 (cfg:20).

Dataset	Credit	Use	Headline
Nuraghe sites (`nurnet_nuraghi.shp`, cfg:14; fields UID, NAME, COMUNE, TWR_N, TWR_FILL in `site_fields`, cfg:50; TYPE read through `REALISTIC.mixed.type_field`, cfg:232, by `code/lit.py:site_types`, lines 173-178)	Nurnet's data — a Nurnet merge and deduplication of various sources	7,167 site points: 5,902 with one tower, 788 with two, 230 with three, 128 with four, 111 with five, 7 with six and 1 with seven; TWR_FILL = 1 (assumed count) at 3,764 sites; TYPE gives the lift rule (688 corridor nuraghi)	yes
Site building stone (field `materiale`; cfg:29, token map cfg:35)	Nurnet's data (Nurnet GeoPortal)	Hybrid stone rule: the recorded stone where known, otherwise the 7 permitted classes. In the mu 0.38 haul table: 2,114 recorded, 5,052 fallback, 1 recorded but unreachable (fallback)	yes (via `cache/`)
Lithology (`geology/sardinia_lithology_25832.shp`, cfg:16; field STONE)	ARPAS 2019, Carta Litologica della Sardegna 1:25,000 (© RAS/ARPAS, CC BY 4.0)	Permitted outcrops (cfg:27), rasterised with the cell-centre rule	yes (via `cache/`)
Elevation (`dem/sardinia_dem_GL030_25832.tif`)	Copernicus DEM GLO-30 (© DLR e.V. 2010-2014 and © Airbus Defence and Space GmbH 2014-2018, provided under COPERNICUS by the European Union and ESA)	Directional drag-cost rasters at mu 0.3, 0.38 and 0.1; land and sea mask and slope raster for the sea tables	yes (via `cache/`)
Soils (1:250,000 soil map, land-capability classes)	Aru, Baldaccini & Vacca 1991	Food catchments (`code/tools/food_capacity.py`) for S12 and S12b; check V_density_stone	no
Rivers (`water/sardinia_rivers_25832.shp`)	Regione Autonoma della Sardegna	Distance to streams for the M3 wet-ground variant (`code/tools/wl_inputs.py`)	no

Derived inputs read by the headline (in `cache/`):

- `sites_haul.csv` (mu 0.3 hybrid table);

- haul/sites_haul_mu0.38.csv, haul/sites_haul_mu0.1.csv;
- site_stone_sets.csv, outcrop_code.tif;
- alliance/kbest.npz, alliance/kbest_mu038.npz;
- cost_mu/ (drag-cost rasters at other friction values);
- sea/sites_haul_sea2kn.csv, sea/people_sea2kn.csv.gz (header only: at 2 kn no people journey by sea beats walking), sea/sea_meta.json.

7. Submodels

7.0 Implementation details (ODD+D)

- **Code and entry point.** Python. code/model.py:main dispatches --alliance to code/alliance.py:run (lines 724-745).
- **repast4py.** SharedContext holds the agents. schedule.init_schedule_runner with a repeating event per season and schedule_stop(max_seasons + 1) (lines 244-245, 562-563).
- **mpi4py.** Seed-parallel, with each model on MPI.COMM_SELF. Results are independent of the number of ranks (code/tests/verify_alliance.py).
- **numba.** The day loop run_days (lines 75-113) and the Dijkstra searches (code/nbcommon.py:multi_source_dijkstra, lines 80-145; code/tools/kbest_quarries.py:reverse_kbest, line 33).
- **Resumable output and run signature.** Per-seed part files go to <cache_dir>/runs/alliance_<scenario>/<signature>/ (alliance.run, lines 2078-2079). A finished seed leaves a .done marker and is skipped on restart. The signature (alliance.signature, lines 2001-2045) is the first 12 hex digits of a SHA-1 over:
  - the configuration blocks hours_per_day, builders, stone, transport, and alliance without list, verify_scenarios and sensitivity (lines 2002-2004);
  - the resolved scenario dict, which includes the work-party rule, population ceiling, hands caps and sea speed (line 2005);
  - the columns UID, TWR_N, haul_flatm and haul_m of cache/sites_haul.csv (line 2006);
  - lit_scenarios.webster_md_per_tower and lit_scenarios.lift_share_of_build (lines 2007-2008);
  - the v11 block and its version string, when webster_day_fix is on (lines 2009-2011);
  - with a sea speed: the site sea-leg columns of the sea table, the sea block and the sea version string (lines 2015-2019, 2024-2025);
  - with tech rules: every lit_scenarios key except list, the chosen rule set and its version string (lines 2020-2023);
  - version strings and inputs of other scenario families, when used (none in the headline; lines 2012-2014, 2026-2039);
  - CODE_VERSION (line 2044).

It does **not** cover the scenarios block of the configuration, for example scenarios.split_cost_mh (cfg:115), which acts on the headline (H13), and scenarios.help_walk_ms (cfg:117), which the sea-table builder reads for people's walks (code/tools/sea_routes.py:472). Nor does it cover the cached k-best and friction tables: alliance/kbest.npz, alliance/kbest_mu038.npz, haul/sites_haul_mu0.38.csv and haul/sites_haul_mu0.1.csv. Runs are additionally separated by scenario name (folder alliance_<scenario>) and by cache_dir. A change to an unhashed input under the same scenario name and cache_dir would therefore reuse stale part files. **This omission is a known limitation, scheduled to be fixed in the next code version.**

- **Headline run.** cache/runs/alliance_MOST_REALISTIC_AUDITED_LIT_P10/ of the run holds two signature folders of 100 seeds each. This description, and every result of the headline, refers to d01216d14207 (run log of the chain: second launch of the headline, 16:01-16:07 on 2 November 2026, after the sea tables were rebuilt). Its sea tables were built at 0.27 m/s drag and 1.39 m/s walk (7.1, item 6). The other folder, cc1399a1a252 (15:51-15:57), comes from the first launch, made on the earlier sea tables built at 0.6 m/s. It was discarded. The two signatures differ because the sea-leg columns are hashed. The per-scenario outputs of the headline were written from d01216d14207.
- **Software versions.** The full run used these versions, read from the run's environment: Python 3.13.5, repast4py 1.3.1, mpi4py 4.1.2 (on Open MPI 5.0.7), numpy 2.5.4, pandas 3.0.6, numba 0.68.0, scipy 1.18.1, rasterio 1.5.2, geopandas 1.2.0 and PyYAML 6.0.3. The same versions are pinned in notebooks/requirements-colab.txt. code/requirements.txt gives the minimum versions: Python 3.10+, repast4py >= 1.1.6, mpi4py >= 3.1, numpy >= 1.24, numba >= 0.58, pandas >= 2.0, rasterio >= 1.3, geopandas >= 0.14, pyogrio >= 0.7, PyYAML >= 6.0.
- **Code location.** The code and configuration described are in this repository: code/ (the run's code, v13 with the social switches off by default) and config/config_full_audited.yaml; this description is ODD/ODD.md.
- **Memory.** About 6 ranks fit the 8 GB container of the full run.

7.1 Preprocessing: cost surface, stone rule, routes and faces (run once)

1. **Drag cost** (code/tools/build_cost_mu.py:band_cost, lines 24-34; recipe as in costsurface/build_drag8.py). For a move of horizontal length d (30 m, or 42.43 m on the diagonal) with gradient t = clip((z_B - z_A)/d, -1, 1), cost = max(0.5, |mu + t| / mu) x d, rounded to an integer (np.rint). The unit is flat-metre equivalents. The delivered raster has mu = 0.3; rasters at mu 0.38 and 0.1 are rebuilt with the same recipe. Band k is the cost into neighbour k (N, NE, ... NW); 0 means impassable.
2. **Outcrops** (code/preprocess.py:run_rule, lines 198-318). Lithology polygons are rasterised onto the cost grid with the cell-centre rule (rasterize_all_touched: false, cfg:47). Only polygons with BUILDSTONE == 1 are rasterised (lines 215-221), so the permitted classes are those of allowed_stones (cfg:27) that are also BUILDSTONE classes.
3. **Hybrid stone rule** (stone_rule: hybrid, cfg:26; materiale_sets, lines 170-195). A site whose recorded stone maps to one or more classes searches those classes; otherwise it searches the 7 permitted classes (cfg:27). One multi-source forward Dijkstra (code/nbcommon.py:multi_source_dijkstra) runs per distinct class set. It gives, for every cell, the least drag cost from any permitted outcrop cell (haul_flatm), the source cell (quarry location and stone) and the path length (haul_m). Recorded stone with no reachable outcrop falls back to the 7-class search.
   - Snapping: sites on nodata or unreachable cells are moved to the nearest valid cell within 20 cells (cfg:51).
   - Unreachable sites: they get a local quarry (cfg:55; applied without reading the key at lines 300-305).
   - In the tables used, no site is snapped and none is unreachable (Appendix D, D6-D7).
   - The same is run with the mu 0.38 and mu 0.1 rasters (haul/sites_haul_mu0.38.csv, haul/sites_haul_mu0.1.csv).
4. **k-best faces** (code/tools/kbest_quarries.py:main, lines 92-187).
   - Regions are the 8-connected components of each stone class.
   - For each site, a reverse directional Dijkstra on the mu 0.3 raster settles cells in order of drag cost to the site until k_quarries = 5 distinct regions of the site's permitted classes are found (cfg:252), or kbest_max_settle = 3,000,000 cells (cfg:253).
   - The search starts at the snapped cell (line 134-135).
   - Each face's path is stored move by move, with its costs at mu 0.3 and mu 0.1.
   - Face 0 equals the multi-source route of sites_haul.csv.
5. **Re-costing at mu 0.38** (code/tools/kbest_mu038.py:main, lines 23-65). The same paths are re-costed with the mu 0.38 raster. The geometry is not re-optimised.
6. **Sea tables** (code/tools/sea_routes.py:main, from line 191; parameters in sea_params, lines 154-171).
   - Water: boat routing on 90 m blocks (sea_grid_factor 3, cfg:357; a block is water if at least 50% of its cells are sea) inside a 2 km coastal band (cfg:358).
   - Landings: beach cells (z <= 10 m and slope < 10 deg, cfg:361-cfg:362) or hoist cells (z <= 5 m, cfg:363), attached to water within 2 blocks (cfg:364).
   - Eligibility: sites and quarries within 2 km of the coast (cfg:359); land legs computed within 5 km of the coast (cfg:360).
   - Route choice: a three-stage Dijkstra (land, sea, land) in crew-hours. The site is in sea mode when this beats its land route.
   - The tables of the full audited run were built with drag speed 0.27 m/s and walk 1.39 m/s (cache/sea/sea_meta.json). People's sea journeys are tabulated only when they beat walking and take at most 6 h (cfg:370); at 2 kn there are none.

Rationale. Least-cost hauling from the nearest permitted outcrop reflects the Sardinian view of local, least-cost stone (Webster 2001: 122-123; Gallin 1989 via Webster 1991: 848). The hybrid rule is specified by the authors. The slope term follows F = m g (mu cos a + sin a) (Boswinkel 2021: 43-44); the 0.5 floor on downhill moves is an assumption.

7.2 Technique rules and stage labour per site

Rule per site (code/lit.py:mixed_haul, lines 181-196; resolve_site, lines 199-213; settings REALISTIC, cfg:221). All rules are deterministic and identical in every seed.

Friction:

- Long rule (tallowed rails, mu 0.1, cfg:231): when the site's route in the mu 0.38 table is longer than long_min_haul_m = 1,000 m (cfg:229) and the site is reachable in the mu 0.1 table. That row's values (quarry, stone, on-outcrop) then replace the mu 0.38 row. This gives 786 sites.
- Short rule (sledge on dry ground, mu 0.38, cfg:230): all other sites.

Quarrying:

- on the outcrop (on_outcrop; 4,940 sites): joints_low;
- otherwise: scree (cfg:225).

Dressing:

- TWR_N <= 1: none;
- TWR_N >= 3: fine facing;

- TWR_N = 2: Webster's rate (cfg:227-cfg:228).

Lifting:

- lever for TYPE "Nuraghe a corridoio (protonuraghe)";
- internal stair otherwise (cfg:233-cfg:234).

Density and learning. Density and hardness are on; within-site learning (S8) is on.

Stage labour (`code/lit.py:stage_hours`, lines 92-139, called with learning off at `code/alliance.py:423`). For a site with T towers and n_p pieces:

- V = 1,350,000 x T / 2,386 m³ of stone. The block volume is held fixed; density changes only the mass.
- **Quarry** = r_q x V x h, with r_q = 11.1 ph/m³ (`joints_low`) or 2.0 ph/m³ (`scree`) (cfg:147). The hardness h is 2.0 for basalt and granite and 1.0 otherwise (cfg:183).
- **Dress**:
  - none: 0;
  - Webster: 1,300 md x 8 h x T x h;
  - fine: (1,300 x 8 x T + A x 0.5 x T x (131.2 - 40.95)) x h. A = pi (R0 + R1) sqrt(H² + (R0 - R1)²) = 420.5 m² is the outer face of one tower, with R0 = 6 m, R1 = 3.4 m, H = 14 m (cfg:158, cfg:154, cfg:155).
- **Build** (setting) = 700 md x 8 h x T x (1 - 0.5) = 2,800 ph x T (cfg:142, cfg:144). The day lengths come from `code/lit.py:md_hours` (lines 79-89); dress and build use 8 h (cfg:94).
- **Lift**, from the cone course weights w_k (`cone_course_weights`, lines 33-44). These are the stone shares of the 16 courses of a truncated cone, 12 m to 6.8 m over 14 m: 15 courses of 0.9 m and a top course of 0.5 m, because the course edges are capped at the tower height (`code/lit.py:37-38`); w_0 = 0.0993.
  - Stair: n_p x (1 - w_0) x 0.37 x (1.0 / 0.9) = 0.370 ph per piece (cfg:160).
  - Lever: n_p x sum_k(k w_k) x 0.37 = 2.179 ph per piece (cfg:159).
- **Per-tower values** `site_stage` = quarry/T, dress/T, (build + lift)/T (lines 432-435).

Mass factor. mf = rho(stone) / 2,386 (cfg:182; `code/lit.py:mass_factor`, lines 64-69). Densities are 3,000 (basalt), 2,700 (granite), 2,600 (limestone/marble) and 2,750 (metamorphic) kg/m³. The other quarry stones (acid volcanic, andesite, schist, tuff, trachyte; 2,047 sites) keep mf = 1 and hardness 1. The stone used is the one in the site's mixed haul table, whichever face is later chosen (Appendix D, D13-D14).

Rationale. Each rule takes the option best supported for that kind of site: on-site quarrying along joints at Duos (Webster 1991: 848); collected stone (Lilliu & Zucca 2005: 41); fine masonry on complexes (Contu 2002: 29-32); the intramural stair as the builders' ramp (Laner 1999: 30). The thresholds are assumptions (Appendix A).

7.3 Work party and population ceiling

`code/alliance.py:_population_ceiling` (lines 565-594) and `code/lit.py:draw_residents`, `residents_range`, `builders_of` (lines 230-244).

1. **Residents.** r_i ~ U[lo, hi] from the row TWR_N of `residents_by_towers` (7 and more towers use the last row), as the first uniform of the site's stream.
2. **Cohort sums.** The residents are summed within each P10 cohort (7.11). The peak is the largest cohort sum.
3. **Redraws.** If the peak exceeds `population_ceiling` = 300,000 (cfg:393), all residents are redrawn from `default_rng([seed, 31337, k])` for k = 1, 2, ... up to 100. After that they would be scaled down in proportion. The block streams are untouched.
4. **Builders.** B_i = max(1, floor(`adult_share` x r_i + 0.5)) with `adult_share` = 0.4 (cfg:385).
5. **Record.** `pop_info` is written per seed. In the 100 seeds of the headline run the ceiling never binds: the largest active peak is 32,681 residents (per-seed `population.json` files).

Rationale. The work party scales with towers, is drawn at random and rests on published population figures (rule specified by the authors). The adult share is Webster's men available (population/5, Webster 1991: 854, Table 2 n. 4) plus an equal number of women. The ceiling is Calò et al. (2008, p. 41), used as a ceiling.

7.4 Blocks, crews and splitting

`AllianceModel.__init__`, lines 337-446.

Blocks:

1. Target mass: 1,350,000 kg x T x mf (cfg:67).
2. Draws: lognormal with median 450 x mf kg (cfg:68) and sigma 0.6 (cfg:69), clipped to [60 mf, 2,400 mf] kg (cfg:70-cfg:71), in chunks of int(target / 450) + 16.

3. Truncation: drawing stops when the cumulative mass reaches the target; the block list is cut at the first block where it does.
4. Towers: block j goes to tower min(T - 1, floor(mass before j / per-tower mass)).

Crews. The pull per person is pull_i = `f_person_kgf` / mu_i = 30 / 0.38 = 78.9 kg on dry ground, or 30 / 0.1 = 300 kg on rails (cfg:173; `code/lit.py:transport`, lines 142-151).

Splitting (`pieces`, lines 397-410). A block whose crew ceil(m / pull_i) exceeds the work party B_i is cut into the fewest equal pieces whose crew fits. Each cut adds `split_cost_mh` = 1 man-hour (cfg:115) to the tower's quarry task.

Per tower (`tech_t`, lines 412-414):

- S_site = sum over pieces of ceil(m_p / pull_i);
- S_038 = the same at 78.9 kg;
- S_03 = the same at `pull_kg_per_person` = 100 kg (cfg:73).

Sea faces. Pieces must also fit the 2,000 kg craft payload (cfg:365). Lift labour is added for the extra pieces (lines 437-446).

Rationale. Block size and mass are Webster's (1991: 852). The crew rule is people = mass / pull (Boswinkel 2021: 52, the 300 N peak). Splitting heavy blocks to fit a small party has no source (assumption).

7.5 Transport labour

`trip` (lines 1363-1373), `leg_mh` (lines 1375-1392), `_leg_mh_tech0` (lines 1399-1454); `code/sea.py:trip_h`, `params`, `hoist_ph_per_block` (lines 21-43).

Trip time per crew member, in hours:

- whole route: trip = max(C, 100) / 0.27 / 3600 + max(L, 100) / 1.39 / 3600 + 0.25 (cfg:76, cfg:78, cfg:79, cfg:80);
- relay leg: C / 0.27 / 3600 + L / 1.39 / 3600 + 0.125, with no minimum haul.

Cost and length on a land face:

- C = the sum of the route's move costs at the site's friction (`mv_c038`, or `mv_c01` for the long rule), in flat-metre equivalents;
- L = the face length for the whole route, or the sum of the move lengths for a leg;
- a face with no route (local stone) uses its C and L for the whole route.

Transport man-hours = S_site x trip.

Sea face: man-hours = S_038 x land_h + S_03 x boat_h + n_p x hoist_ph, where:

- land_h = ((max(hf1, 100) + max(hf2, 100)) / 0.27 + (max(hm1, 100) + max(hm2, 100)) / 1.39) / 3600 + 2 x 0.25;
- boat_h = boat_factor x 2 x sea_km x 1000 / (2 kn x 0.514444) / 3600, with boat_factor = crew x pull / payload = 10 x 100 / 2,000 = 0.5 (cfg:366), i.e. 0.27 h per km of sea route;
- hoist_ph = (0.37 / 0.9) x hoist height. No sea-mode site at 2 kn lands at a hoist.

Rationale. Crews drag at a sustained speed and walk back empty. The drag speed is de Haan's (2009: 7) power-based value; the walk is Tobler's (1993) flat-ground speed. The craft analogue is from Micronesia (Hazell & Fitzpatrick 2006: 12, 18).

7.6 Tower start: decisions

`start_tower` (lines 1522-1573) and `choose` (lines 1306-1346).

Decision dimensions (requests on, no pre-existing ties):

- `quarry_face`: the indices of the site's faces;
- `req_quarry`, `req_dress`, `req_build`: 0 or 1 each;
- `relay`: 0, 0.25, 0.5 or 0.75 (cfg:261).

Prior. On the first tower the `quarry_face` table is filled with -(transport man-hours of the whole route + the site's per-tower stage labour + `setup_mh`) / tonnes (lines 1547-1552).

Choice. Draws come from the community's own decision stream:

1. **Imitation.** With probability `p_imit` = 0.3 (cfg:275; `imitation` true), the community looks at the last strategy of every neighbour within R. It takes the one with the lowest cost per tonne, if that is lower than its own last cost (or if it has none yet). It copies each dimension whose value is valid for itself; a quarry face is copied only if the neighbour's

outcrop region is among its own faces.

2. **Exploration.** For each remaining dimension, with probability epsilon = 0.1 (cfg:273) it picks a uniformly random value.
3. **Exploitation.** Otherwise it picks the value with the highest Q. Untried values count as 0, and ties are broken uniformly at random.

Sea face. If the chosen face is the sea face and the tower has payload-split values, those are used (lines 1556-1562).

Rationale. Communities "search for the most optimal way to build" (aim specified by the authors). The update is the standard bandit form (Sutton & Barto 2018: 28, 32, 34). The rates are assumptions.

7.7 Tasks of one tower, help requests and joint work

`start_tower` (lines 1574-1713), `perform` (lines 1576-1609), `request` (lines 1187-1230), `_candidates_R` (lines 1291-1303), `_join_split` (lines 1480-1490), `_join_task` (lines 1492-1504), `_perform_join` (lines 1506-1520), `_hands_cap` (lines 1463-1478).

Task chain. The tasks are created in order. Each is downstream of the previous one:

1. **Quarry**, performed by the owner or with a helper (`req_quarry`). Labour = per-tower quarry labour x discount (7.9). On top, without discount: `setup_mh` = 200 man-hours (cfg:278) if the owner has not opened that region before (`shared_quarry` is false, so an ally's face is not free), plus the splitting man-hours.
2. **Dress** (`req_dress`), at the quarry.
3. **Transport.**
   - A relay is possible when relay > 0, the face has a route, its mu 0.3 cost C >= `relay_min_flatm` = 300 (cfg:262) and a hand-off exists.
   - The hand-off h is the first move at which the cumulative mu 0.3 cost reaches f x C (`Static.__init__`, lines 158-163).
   - The owner asks for leg 1 among candidates nearer the quarry than itself (straight line).
   - If one accepts, leg 1 (moves a..h) is a joint task and leg 2 (h..b) is done by the owner alone. Otherwise the owner hauls the whole route.
4. **Build and lift** (`req_build`), at the site. This is the final task; its labour = (build + lift) per tower x discount.

Candidates (`_candidates_R`). Neighbours within `R_m` = 10 km (cfg:260), sorted by:

1. skill factor for the stage (ascending: most experienced first, because `specialisation` is on);
2. the asker's trust in them (descending);
3. distance;
4. id.

At most `max_asks` = 3 (cfg:263) are asked, in turn.

Acceptance (`request`). The asked community j accepts asker i with probability sigmoid(beta0 + beta1 T(j, i) - beta2 backlog_j - beta3 d_ij / 1000), where:

- the betas are 0, 2, 0.05 per day and 0.2 per km (cfg:268-cfg:271);
- backlog_j = (remaining labour of j's active tasks) / (B_j x 8 h) (`backlog_days`, lines 1122-1127);
- the draw comes from j's stream. Defectors would always refuse; there are none.

Trust update (`trust` true). On acceptance, T(i, j) <- min(1, T(i, j) + 0.1). On refusal, if j has received help from i before, T(i, j) <- max(0, T(i, j) - 0.2) (cfg:264-cfg:266). An acceptance records (i, j) in `received_from`. The first season of mutual trust >= 0.7 (cfg:267) is recorded. The first acceptance ends the asking; if all refuse, the owner works alone.

Join mode (`help_mode: join`). The helper adds hands to the owner's stage:

- helper share f = h / (B_i + h), with h = max(0, min(B_j, cap - B_i));
- the hands cap (`join_hands_per_block`, cfg:394):
  - quarry: 4 x pieces of the tower;
  - dress: 2 x pieces;
  - build: 6 x max(1, pieces / 16 courses);
  - leg 1: the sum of the pieces' crews;
- the owner's task gets MH x (1 - f). The helper's task gets MH x f + walk, where walk = B_j x d / 1.39 / 3600: the helper's whole party, one way, d = straight-line distance from j to the quarry (quarry, dress, leg 1) or to the site (build);

- both parts are upstream of the next task (`t_up2`);
- the discount (7.9) is that of the main performer: the helper if f > 0.5, else the owner;
- that performer gets one unit of experience.

Rationale. Asking outside one's own site and reciprocity-based trust are the mechanisms specified by the authors. The relay is the PI's example of neighbours dragging stone closer. "Join" replaced "handover" after handover slowed large sites (modelling agent's design choice, accepted by the authors). The logistic form, the betas, the trust steps, R and the number of asks have no published source (Appendix A).

7.8 Daily work

`run_days` (lines 75-113; numba). For each day:

1. **Capacity.** Each performer of an active task gets `capleft` = B x 8 person-hours.
2. **Work.** Tasks are visited in the order (stage index, task id); finished tasks are skipped. A task's allowed work is frac_up x total - done, where frac_up = min(done/total) over its upstream part(s), or 1 if it has none. It takes min(allowed, capleft[performer]).
3. **Completion.** A task is complete when its remainder is <= 1e-9 x total; its `finish_day` is recorded.
4. **Return.** The loop returns at the end of the first day on which a tower's final task completed.

So a community spends its daily hours first on upstream stages, across its own and helped towers, in creation order. Downstream stages proceed in parallel, capped by the share done upstream. There is no idle accounting: labour is the task totals.

Rationale. In the baseline model, builders were never idle (716,700 webster site-runs, module docstring). A tower's labour therefore equals transport plus stage labour, and its duration follows from capacity and the upstream cap.

7.9 Skill and within-site learning

- **Skill.** skill(j, s) = (1 + cum[j, s])^(log2 b), with b = 0.85 (`alliance.defaults`, cfg:281; not overridden by `MOST_REALISTIC`), for s in quarry, dress, build (`skill`, lines 1117-1120).
- **Discount.**
  - Performed by the owner: min(skill(i, s), 0.85^t), where t is the tower index and the S8 rate is `learning_rate` = 0.15 (cfg:189) (lines 1597-1599; join mode: lines 1512-1514).
  - Performed by a helper (main performer): skill(j, s).
- **Experience.** Each credited task adds 1 to cum. With `specialisation` true, help given also counts.

Rationale. Wright's (1936) curve gives the form; 0.85 is the midpoint of his 0.80-0.90 range. The S8 rate has no source; never applying both discounts avoids counting the same learning twice.

7.10 Tower completion and value learning

`finish_tower` (lines 1727-1750):

1. **Reward.** own = the change in `mh_own` over the tower; r = -own / tonnes.
2. **Update.** For each chosen dimension: Q(a) <- Q(a) + 0.5 (r - Q(a)) if a has a value, else Q(a) <- r (cfg:274).
3. **Bookkeeping.** `last_strategy` is set and `tower_idx` advances.
4. **Next.** The next tower starts at once. After the last tower, `finish_day` and `finish_season` are set.

7.11 Phasing and the season loop

Cohorts (`code/phasing.py:cohorts`, lines 22-27). The permutation `default_rng([seed, 4242]).permutation(N)` is cut at floor(k N / 10), k = 0..10, giving 10 cohorts, each sorted by gid.

Starts (`season_step`, lines 1775-1783). Cohort k + 1 starts at the first season start after every tower of the earlier cohorts is complete. Its communities start in gid order with `start_day` = the current day and `start_season` = the current season. Any remaining days of the season in which the last tower of a cohort finished pass without work (line 1804).

Stop. The run stops when all communities have started and finished (lines 1819-1821). It asserts that no tower is unfinished at the safety stop (line 1840).

Rationale. Gradual building in cohorts of 10% follows the authors' specification of phasing. Membership is random because no chronology of individual sites is used (assumption).

7.12 Accounting

charge (lines 1144-1173):

- Each task's labour mh (for a helper's task, including its walk) is charged at creation. The whole mh goes to `mh_total` of the nuraghe it is spent on (line 1147) and to `mh_own` (performer = owner) or to `mh_received` (owner) and `mh_given` (helper) (lines 1149-1153). The walk goes to `mh_walk` (line 1148). Only mh - walk goes to the stage account (`mh_quarry`, `mh_dress`, `mh_transport`, `mh_build` / `mh_lift`, `mh_track`; lines 1162-1173).
- Build labour is split between `mh_build` and `mh_lift` by the site's lift share (accounting only).
- Setup man-hours are moved from `mh_quarry` to `mh_setup` (lines 1624-1625).

7.13 Edge cases and inactive branches

1. **Accepted help that adds no hands.** If the hands cap is not above the owner's party, f = 0: the helper contributes no labour, but the request still counts as accepted (trust rises, `received_from` is set) (`_join_split`, `request`).
2. **Walk for the whole party.** The helper's walk is charged for its whole party B_j even when only h < B_j hands count in the split (`_join_task`).
3. **Very small parties.** Single towers can have B = 2 (r = 5). On dry ground every piece then weighs at most 157.9 kg, so nearly every block is split, at 1 man-hour per cut.
4. **Idle days between cohorts.** A cohort waits for the next season start (7.11).
5. **Faces.** Two sites have only one face. The local-stone face (route -1) is never used: every site has at least one route.
6. **Inactive preprocessing branches.** Snapping and the unreachable policy act on no site in the tables used (Appendix D).
7. **Sea.** People never travel by sea at 2 kn (the people table is empty), so `wdist` is the straight-line distance. Sea faces exist at 46 sites; none uses a hoist landing.
8. **Inactive switches.** Trackways (`tracks`), shared faces (`shared_quarry`), defectors, pre-existing ties, brokers and every v12, gaps and wider-literature switch are off. Their code paths are skipped by guards (`self.cz is None`, `hp is None`, `self.masons` false, `self.units is None`). The v13 social switches (`reputation`, `partner_choice`, `tie_imitation`, `social_log`) are off too, and the v13g `reputation_rule` (default `all`) acts only through them; their code is skipped by the guard `self.soc is None`, and with them off the outputs are identical to the frozen code (7.15).
9. **Numerical guards.** `sigmoid` returns 0 for arguments below -700 (line 117). Run signatures include code-version strings, so part files from older code are never reused (lines 36-55, 2044). The signature does not cover every input (7.0).
10. **Relay geometry.** The hand-off position and the 300 flat-m minimum use the mu 0.3 cost, while the legs are costed at the site's own friction. The route geometry is the mu 0.3 path for every friction (7.1, item 5).

7.14 Random streams

Stream	Draws
`default_rng([seed, uid_int])` per site	residents (first uniform; with the ceiling, a dummy uniform keeps the stream aligned), then block masses (`code/alliance.py:325-356`)
`default_rng([seed, uid_int, 7])` per community	imitation, exploration, tie-breaks of its own decisions; acceptance draws when it is asked (line 462)
`default_rng([seed, 99991])`	start order (unused under P10), defectors (none) (lines 463-464, 487-490)
`default_rng([seed, 4242])`	cohort permutation (`code/phasing.py:22-27`)
`default_rng([seed, 31337, k])`	resident redraws under the ceiling (never used in the headline run)

Seeds 1-100 (`seeds`, cfg:98; `code/model.py:parse_seeds`, lines 496-503). Every random stream is keyed by seed and site or role, so results do not depend on the number of ranks.

The v13 social switches and the v13g reputation rule (7.15) make no draws. A social refusal overrides the acceptance draw already taken from the asked community's stream, so every stream stays aligned with the switches-off run.

7.15 Social mechanisms (v13; off in the headline)

`_init_social`, `_bad`, `_social_gate`, `_partner_order`, `_imit_pool` (lines 1012-1088); calls in `AllianceModel.__init__` (lines 502-504), `charge` (lines 1160-1161), `request` (lines 1205-1206), `candidates` (lines 1276-1279), `choose` (lines 1314-1322) and `start_tower` (lines 1570-1572); output columns in `results` (lines 1886-1891, 1924-1926);

signature (lines 2040-2043). v13g reputation rule: `reputation_rule`, `_rule_tag`, `_sig_sc` and the constants `REPUTATION_RULES`, `REPUTATION_RULE_VERSION` (lines 2146-2178, appended at the end of the file so that no earlier line moved); one-line calls in `_init_social` (line 1027), `_social_gate` (line 1057) and `signature` (lines 2005, 2041). Three opt-in scenario keys, each off unless set in the scenario, plus one rule key (`reputation_rule`) that acts only with `reputation`; none adds a numeric parameter. They are used only by the five v13 and the three v13g scenarios (Appendix B) and only with the v13 code.

Reputation (`reputation: true`).

- **Log.** Every decision of an asked community j on a request of i is logged in j's act log: a help as good, an unjustified refusal as bad.
- **Standing seen by an observer.** Observer k judges j by j's most recent logged act that k observed. k observes an act if k is its requester or lies within R of that requester (straight line; R = the run's interaction radius `S.R`, i.e. `R_m` = 10 km). With no observed act, j is in good standing (`_bad`).
- **Shunning.** A non-defector j that sees requester i in bad standing declines (`_social_gate`). The decline is justified and is not logged, so the shunner keeps its standing (standing rule). A refusal by a defector is logged as bad unless it is a partner-choice refusal (below), which is justified for every community.
- **Rule** (`reputation_rule`; v13g). Which refusals are logged as bad acts. `all` (default; the five v13 runs): every unjustified refusal, as above, including ordinary refusals for backlog, distance or capacity. `gentle`: an unjustified refusal is logged as bad only if the refuser is a defector or has received help from the requester (`(j, i)` in `received_from`, the same condition as the `delta_down` trust update of 7.7 and as cheating under partner choice); any other refusal is not logged at all, so it neither costs nor restores standing (line 1057). Helps and justified refusals are treated as under `all`. Any other value is an error (`reputation_rule`, lines 2162-2167). The key has no effect unless `reputation` is on.
- **Trust.** The trust update of 7.7 sees the gated decision: if the shunner had received help from the asker, the asker's trust in it falls by `delta_down`, as for any refusal. The ask and refusal counters also count social refusals.

Partner choice (`partner_choice: true`).

- **Cheating.** A community j that refuses i without justification after having received i's help (`received_from`) has cheated i. The pair is cleared when j later helps i.
- **Candidate order** (`_partner_order`). The candidate list (7.7) is reordered stably into three groups: first exchange partners (j has helped i, or i has helped j), then the others, and last the communities that cheated i (this group takes precedence over the first). The existing order is kept inside each group, and at most `max_asks` are asked, as before.
- **Helper side** (`_social_gate`). An asked community declines a requester that cheated it. Such refusals are justified: they create no cheating pair and no bad standing.

Imitation along help ties (`tie_imitation: true`).

- **Pool** (`_imit_pool`). In the imitation step (7.6, item 1) the pool is the communities linked to i by help given or received so far (mh > 0, either direction, recorded in `charge`), ordered by distance then id. These may lie beyond R.
- **Fallback.** A community with no such tie uses its neighbours within R, as with the switch off.
- **Choice.** Unchanged: the model with the lowest last cost per tonne, if that cost is below the community's own.
- **Edge case.** In join mode a helper whose hands are capped to 0 still has its walk charged (7.13, item 2), so it has mh > 0 and counts as a tie.

Bookkeeping (`social_log: true`, or any switch on). `sites` gets the `soc_*` columns (`soc_n_shunned`, `soc_n_shun_given`, `soc_n_bad`, `soc_n_good`, `soc_n_pc_refused`, `soc_n_pc_refuse_given`, `soc_n_cheat`, `soc_bad_final`, `soc_help_ties`); `strategy` gets `gid`, `imit_src` (-1 = none) and `imit_tie` (0/1). `social_log` alone changes no behaviour.

Signature and identity. When a social key is set, the run signature also hashes `SOCIAL_VERSION` and `cfg['scenarios']` without its `list` (lines 2040-2043); `CODE_VERSION` is unchanged, so every earlier scenario keeps its signature. A non-default `reputation_rule` appends `REPUTATION_RULE_VERSION` and the rule to `SOCIAL_VERSION` (`_rule_tag`, line 2041), and an explicit `reputation_rule: all` is dropped from the hashed scenario (`_sig_sc`, line 2005), so the rule `all` leaves every signature as it was (the five v13 runs and their comparators were re-checked: unchanged). With every switch off, the v13 code reproduces the frozen code exactly: on `code/sites300_real.txt`, seeds 1-2, the decompressed part files of `MOST_REALISTIC_AUDITED_LIT_P10`, `A5_full_freerider`, `P_D7_G20_kmesh4_trust` and `MOST_REALISTIC_S20_lintels_3t0_P10` are identical (combined hashes in `CODE_PROVENANCE.md`). After the v13g edit, the v13 code before and after the edit gives identical decompressed part files (same subset and seeds) for `MOST_REALISTIC_AUDITED_LIT_P10` and `A5_full_freerider` with every switch off, and for three test scenarios with `reputation` on and the rule `all` (A5_full_freerider + reputation; + reputation + partner choice; the headline + all three switches).

Rationale.

- Boyd & Richerson (2022: 189-190, 194) identify third-party shunning and language-borne reputation as the mechanism that dyadic reciprocity lacks. Kasper & Borgerhoff Mulder (2015: 704, 713) report public, third-party sanctions on refusal.
- Image scoring and standing are the standard formal versions (Nowak & Sigmund 2005; Panchanathan & Boyd 2004).

- The gentle rule follows the form of standing (only unjustified refusals cost reputation; Panchanathan & Boyd 2004) and of reciprocity-based help that is owed by recipients and contingent on need and affordability (Fafchamps 1992: 148); the sanctioned breach is the refusal to reciprocate (Boyd & Richerson 2022: 179; Kasper & Borgerhoff Mulder 2015: 713).
- Apicella et al. (2012: 500) and Kasper & Borgerhoff Mulder (2015: 708-711) show that cooperators assort and that help follows reciprocal partners.
- Apicella et al. (2012: 500) show that behavioural similarity follows social ties more than geography.
- All of these are forager or farmer studies and transfer as form only. The specific rules (observation within R, a binary image set by the last observed act, the three-group order, the exclusion of cheaters, cumulative undirected help ties, and the gentle rule's ingratitude condition) are the authors' idea. They have no numeric parameters (Appendix A; `param_audit/PARAM_AUDIT.md`, rows A10-A13).

Appendix A. Parameters acting on the headline run

Status labels are copied exactly from the parameter audit: `param_audit/PARAM_AUDIT.md` (70 rows) and `param_audit/PARAM_AUDIT_ADDENDUM.md` (6 keys). Citations are those found by the audit, shortened. † marks a page still to be confirmed.

"Code" names the function where the value acts. `cfg:` is the configuration line. **Acts?** says whether the value changes headline results. P4 is listed because it is an audit row, but it does not act. Values marked "no effect" are read by the code but cannot change the headline (Appendix D, D11-D12).

A.1 Audited parameters (70 rows of PARAM_AUDIT) and the 6 addendum keys

ID	Key (cfg line)	Value	Unit	Meaning as implemented (code)	Published range	Status	Citation	Acts?
P1	`mr_work_party.residents_by_towers` (cfg:384)	[5,40], [40,110], [110,185], [185,260], [260,330], [330,400], [330,400]	persons	Residents ~ U[lo, hi] by TWR_N (`lit.draw_residents; alliance._population_ceiling`)	anchors 5-6 per household to about 400 per site	DERIVED	Webster 1991: 853-854; Webster 2015: 51; Webster & Webster 1998 (Duos about 70); Porcarzos about 400 (Webster 1991: 853); interpolation ours	yes
P2	`mr_work_party.adult_share` (cfg:385)	0.4	fraction	Builders = max(1, floor(0.4 r + 0.5)) (`lit.builders_of`)	0.2-0.6	DERIVED	Webster 1991: 854, Table 2 n. 4 (men = population/5), x 2 for adult women	yes
P3	`population_ceiling` (cfg:393)	300,000	persons	Ceiling on residents of the cohort building at once; redraw or scale (`alliance._population_ceiling`); never binding	100,000-700,000 (PARAM_AUDIT P3)	SOURCED	Calò et al. 2008: 41	yes (non-binding)
P4	`scenarios.household_size` (cfg:114)	5	persons	Households of non-cooperators (`model.Site`)	5-6	SOURCED	Webster 1991: 852-854; Webster 1996	no
T1	phase fraction (`_P10`)	0.1	fraction of sites	10 cohorts; next starts when all earlier are built (`phasing.split_name, phasing.cohorts, alliance.season_step`)	none	Authors' assumption (specified by the authors)	no literature sets cohorts; context Webster 1991: 853; Ialongo 2018: 29	yes
T2	cohort membership	random per seed	-	`default_rng([seed, 4242]).permutation (phasing.cohorts)`	none	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source	yes
T3	`alliance.season_days` (cfg:255)	90	days/season (= 1 year)	Working days per tick (`alliance.season_step`)	40-90	Authors' assumption (specified by the authors)	Renfrew 1973: 549 and Erasmus 1965: 280-81 via Webster 1991: 853 (REPORTED-ONLY)	yes

ID	Key (cfg line)	Value	Unit	Meaning as implemented (code)	Published range	Status	Citation	Acts?
T4	hours_per_day (cfg:58)	8	h/day	Daily capacity B x 8 only (AllianceModel.__init__, lines 242, 452). It is not the man-day length of dress and build: with v11.webster_day_fix true (cfg:93) these use v11.webster_day_hours (T5; lit.md_hours, lines 86-88)	5-10	SOURCED	de Haan 2009: 2, 7 (comparative); Boswinkel 2021: 145-146	yes
T5	v11.webster_day_hours (cfg:94)	quarry 5, transport 5, dress 8, build 8	h per man-day	Hours per Webster man-day (lit.md_hours; switch v11.webster_day_fix cfg:93)	-	REPORTED-ONLY	Erasmus 1965: 283 via Boswinkel 2021: 145; Abrams via Boswinkel 2021: 109, 146	dress and build only (= 8 h); quarry and transport: no effect
B1	stone.tonnes_per_tower (cfg:67)	1,350	t/tower	Target stone mass per tower x mf (AllianceModel.__init__ lines 340-341); volume 565.8 m³ at 2,386 kg/m³ (lit.stage_hours)	-	SOURCED	Webster 1991: 852-853, Table 1	yes
B2	stone.block_median_kg (cfg:68)	450	kg	Lognormal median x mf (line 349)	-	SOURCED	Webster 1991: 852	yes
B3	stone.block_sigma (cfg:69)	0.6	log units	Lognormal sigma	0.41-0.72	DERIVED	from the range in Webster 1991: 852	yes
B4	stone.block_min_kg (cfg:70)	60	kg	Lower clip x mf	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	Webster 1991: 852 gives "< 0.06 m³", no lower figure	yes
B5	stone.block_max_kg (cfg:71)	2,400	kg	Upper clip x mf	lintels 1.5-4 t	SOURCED	Webster 1991: 848, 852	yes
B6	lit_scenarios.webster_density_kg_m3 (cfg:143)	2,386	kg/m³	Reference density: stone volume and mass factor (lit.mass_factor, lit.stage_hours)	-	SOURCED	Webster 1991: 852	yes
B7	lit_scenarios.density_kg_m3.basalt (cfg:182)	3,000	kg/m³	Mass factor 3,000 / 2,386 (lit.mass_factor)	2,700-3,300	SOURCED	Lilliu 1955 (peso specifico 3) †	yes
B8	lit_scenarios.density_kg_m3.granite	2,700	kg/m³	Mass factor	2,620-2,760	SOURCED	Sanna & Atzeni n.d., Table 6.2, p. 96	yes
B9	lit_scenarios.density_kg_m3.limestone/marble	2,600	kg/m³	Mass factor	2,420-2,711 (comparative)	Authors' assumption (modelling agent's design choice, accepted by the authors)	Turner 2020: 276-279 (comparative)	yes
B10	lit_scenarios.density_kg_m3.metamorphic	2,750	kg/m³	Mass factor	2,600-2,800	REPORTED-ONLY	textbook values, no page	yes
B11	lit_scenarios.hardness (cfg:183)	basalt 2.0, granite 2.0, limestone/marble 1.0, metamorphic 1.0 (others 1.0)	factor	Multiplies quarry and dress labour (lit.stage_hours)	dressing ratio 1.83	Authors' assumption (modelling agent's design choice, accepted by the authors)	Hurst 1905: 382 via Boswinkel 2021: 120 (dressing, comparative)	yes
Q1	lit_scenarios.quarry_ph_m3.scree (cfg:147)	2.0	ph/m³	Quarry rate off outcrop (lit.stage_hours)	-	SOURCED	Boswinkel 2021: 108, Table 7.2	yes
Q2	lit_scenarios.quarry_ph_m3.joints_low	11.1	ph/m³	Quarry rate on outcrop	-	SOURCED	Boswinkel 2021: 108 (0.09 m³/ph)	yes
Q3	quarry rule (cfg:225)	joints_low on outcrop, scree off	rule	lit.resolve_site	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	context: Webster 1991: 848; Lilliu & Zucca 2005: 41	yes

ID	Key (cfg line)	Value	Unit	Meaning as implemented (code)	Published range	Status	Citation	Acts?
Q4	dress rule (cfg:227-cfg:228)	none at 1 tower; Webster at 2; fine at >= 3	rule	`lit.resolve_site`	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	context: Contu 2002: 29-32; Lilliu & Zucca 2005	yes
Q5	`lit_scenarios.webster_md_per_tower` (cfg:142)	quarry 500, dress 1,300, build 700 (transport 1,100 unused)	man-days/tower	Webster stage labour (`lit.stage_hours`)	-	SOURCED	Webster 1991: 852-853, Table 1	dress and build only; quarry 500: no effect
Q6	`lit_scenarios.facade` (cfg:154)	131.2 / 40.95	ph/m³	Facing increment 90.25 ph/m³ on fine-dressed sites (`lit.stage_hours`)	95.7-166.7 / 6.7-75.2	DERIVED	Boswinkel 2021: 144, Table 8.3 (midpoints)	yes
Q7	`lit_scenarios.facing_thickness_m` (cfg:155)	0.5	m	Depth of the fine facing	-	DERIVED	Webster 1991: 852 (block depth); Lilliu 1955	yes
G1	`lit_scenarios.geometry.tower_height_m` (cfg:158)	14	m	Cone height: 16 courses; facing area; join cap for build (`lit.cone_course_weights`, `alliance._hands_cap`)	10-18	SOURCED	Webster 1991 (pp. 842-847) †	yes
G2	`lit_scenarios.geometry.base_diam_m`	12	m	Cone base	10.75-16	DERIVED	Lilliu 1955: 68; Contu 2002; Scintu 2003: 43; Webster 1991 (median of central values 11.9)	yes
G3	`lit_scenarios.geometry.top_diam_m`	6.8	m	Cone top	5.5-8.1	DERIVED	Contu 2002 batter 8-13 deg: 12 - 2 x 14 x tan 10.5 deg	yes
G4	`lit_scenarios.geometry.course_height_m`	0.9	m	Lift step: course count, stair rule	-	SOURCED	Boswinkel 2021: 124 (lift step)	yes
G5	`lit_scenarios.lever_ph_per_block_course` (cfg:159)	0.37	ph per piece per course	Lever and stair lift; sea hoist 0.37/0.9 per m (`lit.stage_hours`, `sea.params`)	-	SOURCED	Boswinkel 2021: 124	yes
G6	`lit_scenarios.lift_share_of_build` (cfg:144)	0.5	fraction	Webster build x (1 - 0.5) kept as setting; lift half replaced by the stair or lever rule	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source; Webster 1991: 853 (0.8 m³/md includes setting)	yes
G7	`lit_scenarios.stair_lever_last_m` (cfg:160)	1.0	m	Stair: only the last metre per course is levered	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	Laner 1999: 30, 33 (no time)	yes
G8	lift rule (cfg:233)	lever for "Nuraghe a corridoio (protonuraghe)", stair otherwise	rule	`lit.resolve_site`	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	Laner 1999: 30	yes
G9	`lit_scenarios.learning_rate` (cfg:189)	0.15	per tower	S8: tower t costs 0.85^t in craft stages (`alliance.perform`)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no archaeological rate found	yes
G10	`alliance.defaults.b` (cfg:281)	0.85	progress ratio	Wright skill exponent log2 b (`alliance.skill`)	0.80-0.90	DERIVED	Wright 1936: 124-126	yes
H1	`transport.drag_speed_ms` (cfg:76)	0.27	m/s	Loaded drag speed (`alliance.trip`, `sea.params`, sea tables)	0.19-0.355	SOURCED	de Haan 2009: 7 (calculated, comparative); Harris 2016 AAM p. 4 (1 m/s peak only)	yes
H2	`lit_scenarios.f_person_kgf` (cfg:173)	30	kgf	Pull per person = 30/mu (`lit.transport`)	20-46	SOURCED	Richards & Whitby 1997: 237, 242; de Haan 2009: 6	yes

ID	Key (cfg line)	Value	Unit	Meaning as implemented (code)	Published range	Status	Citation	Acts?
H3	transport.pull_kg_per_person (cfg:73)	100	kg/person	Boat-leg crews and boat factor (sea.params); sea-table route choice	-	DERIVED	30 kgf / 0.3	yes (sea faces)
H4	transport.return_walk_ms (cfg:78)	1.39	m/s	Empty walk back; helper walks (alliance.trip, _join_task)	1.31-1.47	SOURCED	Tobler 1993: 3; de Haan 2009: 7	yes
H5	transport.load_unload_h (cfg:79)	0.25	h per trip	Loading and unloading; half per relay leg	0.25-0.33	DERIVED	Boswinkel 2021: 116; de Haan 2009: 7	yes
H6	transport.min_haul_flatm (cfg:80)	100	flat-m	Minimum haul of a whole route	up to 200 m	SOURCED	Boswinkel 2021: 116; Webster 1991: 852	yes
H7	REALISTIC.mixed.short.mu (cfg:230)	0.38	-	Dry-ground sledge: crews 78.9 kg/person, mu 0.38 route costs	0.25-0.5	SOURCED	Boswinkel 2021: 114, 124	yes
H8	REALISTIC.mixed.long.mu (cfg:231); alliance.track_mu (cfg:277)	0.1	-	Tallowed rails: crews 300 kg/person, mu 0.1 route costs (track_mu: tracks off)	-	SOURCED	Richards & Whitby 1997: 239	yes
H9	REALISTIC.mixed.long_min_haul_m (cfg:229)	1,000	m	Rails where the mu 0.38 route exceeds it (lit.mixed_haul)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source	yes
H10	cost raster mu (cfg:15; lit_scenarios.baseline_mu cfg:145)	0.3	-	Route geometry of all faces; relay hand-off and minimum; sea-table land legs	0.25-0.38	Authors' assumption (modelling agent's design choice, accepted by the authors)	Cotterell & Kamminga via Boswinkel 2021: 113 (comparative)	yes
H11	cost recipe (tools/build_cost_mu.py)	max(0.5, abs(mu + t)/mu) x d	-	Directional drag cost	-	SOURCED	Boswinkel 2021: 43-44; Richards & Whitby 1997: 237 (0.5 floor: no source)	yes
H12	alliance.relay_fractions (cfg:261); relay_min_flatm (cfg:262)	0.25, 0.5, 0.75; 300	fraction; flat-m	Relay options and minimum route cost	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	mechanism specified by the authors; values no source	yes
H13	scenarios.split_cost_mh (cfg:115)	1.0	man-hours per cut	Added to the quarry task (pieces, tech_t)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source	yes
H14	join_hands_per_block (cfg:394)	quarry 4, dress 2, build 6	hands per piece	Join cap (_hands_cap)	-	SOURCED	build: Hodges 1989 via Boswinkel 2021: 116, 124; de Haan 2009: 22; quarry: Richards & Whitby 1997: 238 (weak); dress: none	yes (non-binding)
K1	stone_rule (cfg:26); allowed_stones (cfg:27)	hybrid; 7 classes	rule	Recorded stone else 7 permitted classes (preprocess.run_rule)	-	Authors' assumption (specified by the authors)	Webster 2001: 122-123; Gallin 1989 via Webster 1991: 848	yes (via cache)
K2	alliance.k_quarries (cfg:252)	5	faces	k-best faces per site (tools/kbest_quarries.py)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source	yes (via cache)
K3	alliance.setup_mh (cfg:278)	200	man-hours	Opening a face not yet used by the community	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source (plausibility: Webster 1991: 848; Boswinkel 2021: 122)	yes

ID	Key (cfg line)	Value	Unit	Meaning as implemented (code)	Published range	Status	Citation	Acts?
A1	alliance.R_m (cfg:260)	10,000	m	Radius for requests, relays, imitation	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source	yes
A2	alliance.max_asks (cfg:263)	3	candidates	Asks per task	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source	yes
A3	alliance.trust0, delta_up, delta_down (cfg:264-cfg:266)	0.5, 0.1, 0.2	trust units	Initial trust; gain on help; loss on refusal after help received	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	principle specified by the authors; values no source	yes
A4	alliance.beta0..beta3 (cfg:268-cfg:271)	0, 2, 0.05, 0.2	logit; per trust; per day; per km	Acceptance logit	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source	yes
A5	alliance.alliance_threshold (cfg:267)	0.7	trust	Alliance record (no effect on labour or time)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source	output only
A6	alliance.epsilon (cfg:273)	0.1	probability	Exploration per dimension	-	SOURCED	Sutton & Barto 2018: 28 (methodological)	yes
A7	alliance.alpha (cfg:274)	0.5	-	Value step size	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	Sutton & Barto 2018: 32 (form only)	yes
A8	alliance.p_imit (cfg:275)	0.3	probability per tower	Imitation	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	Gilbert 2020 s. 1.3.6 (concept)	yes
A9	help_mode: join, specialisation: true (cfg:392-393)	rule	-	Join mode; help given builds skill	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	PARAM_AUDIT Part 2	yes
S1	sea_kn (cfg:392)	2	knots	Laden craft speed (sea.trip_h; 2-kn tables)	0.54-5	SOURCED	Hazell & Fitzpatrick 2006: 18 (comparative)	yes
S2	sea.payload_kg (cfg:365)	2,000	kg	Craft payload: piece split, boat factor	-	SOURCED	Hazell & Fitzpatrick 2006: 12, 18	yes
S3	sea.crew (cfg:366)	10	persons	Boat factor = 10 x 100 / 2,000	-	DERIVED	Hazell & Fitzpatrick 2006: 18 (800 kg crew) / 80 kg	yes
S4	sea.transfer_h (cfg:367)	0.25	h per landing	Transfer time per landing	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	no source	yes

ID	Key (cfg line)	Value	Unit	Meaning as implemented (code)	Published range	Status	Citation	Acts?
S5	sea.sea_band_m 2,000; coast_band_m 2,000; land_band_m 5,000; beach_max_z_m 10; beach_max_slope_deg 10; hoist_max_z_m 5; people_walk_to_landing_m 2,000; people_embark_h 0.25 (cfg:358-cfg:369)	as listed	m, deg, h	Sea-table geometry (tools/sea_routes.py)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	words specified by the authors; numbers the agent's	yes (via cache)
X1	max_seasons (cfg:393)	20,000	seasons	Safety stop	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	technical	no (never reached)
X2	seeds (cfg:98)	1-100	-	Replicates	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	technical	yes
T-1	sea.sea_grid_factor (cfg:357)	3	cells per block side	Boats routed on 90 m blocks, water if >= 50% sea (tools/sea_routes.py:229)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	technical (addendum)	yes (via cache)
T-2	sea.attach_radius_blocks (cfg:364)	2	blocks	Landing-to-water attachment (tools/sea_routes.py:263)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	technical (addendum)	yes (via cache)
T-3	sea.people_max_h (cfg:370)	6.0	h	Cut-off of people sea journeys (tools/sea_routes.py:483)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	technical (addendum)	no effect at 2 kn
T-4	rasterize_all_touched (cfg:47)	false	-	Cell-centre outcrop rasterisation (preprocess.py:73, 222)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	technical (addendum)	yes (via cache)
T-5	nodata_snap_radius_cells (cfg:51)	20	cells (600 m)	Site snapping (preprocess.py:106, 245; tools/kbest_quarries.py:135)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	technical (addendum)	no effect (no site snapped)
T-6	unreachable_policy (cfg:55)	local_min_haul	-	Local quarry for unreachable sites (preprocess.py:143; applied unconditionally in run_rule, lines 300-305)	-	Authors' assumption (modelling agent's design choice, accepted by the authors)	technical (addendum)	no effect (no unreachable site)

Count: 76 rows: 70 audit rows, of which 69 act on the headline (P4 does not), plus the 6 addendum keys.

A.2 Switches and derived constants of the headline (not parameter rows of the audit)

Item	Value (cfg line)	Effect
requests, trust, specialisation	true (cfg:392)	requests, trust updates, help builds skill and orders candidates
tracks, shared_quarry, defector_frac, track_cost_mh_per_m	false, false, 0.0, 2.0 (cfg:280-281)	off; track cost acts only with tracks
imitation	true (cfg:281)	enables A8
tech_rules	REALISTIC (cfg:392)	per-site rules (7.2)

Item	Value (cfg line)	Effect
REALISTIC.density, REALISTIC.learning	true (cfg:221)	S6 density and hardness; S8
lit_scenarios.fine_min_towers	2 (cfg:156)	gate for fine facing; never binds (REALISTIC requires >= 3)
v11.webster_day_fix	true (cfg:93)	enables T5
sea.extra_speeds_kn	[2] (cfg:356)	builds the 2-kn tables
sea.boat_pull_kg_per_person	absent, so transport.pull_kg_per_person	boat factor 0.5
alliance.kbest_max_settle	3,000,000 (cfg:253)	search cap of the k-best search
alliance.max_ranks	8 (cfg:279)	MPI ranks taking seeds; no effect on results
alliance.start_rate, alliance.max_seasons	1/450, 3,000 (cfg:256, cfg:258)	not used: cohorts replace the start rate; MR sets 20,000
builders.*	50 + 25 per tower etc.	not used (the work-party rule replaces it)
derived: pull at mu 0.38 / mu 0.1	78.9 / 300 kg per person	lit.transport
derived: courses per tower	16 (15 of 0.9 m, top course 0.5 m)	ceil(14 / 0.9)
derived: lift per piece, stair / lever	0.370 / 2.179 ph	7.2
derived: facing area per tower	420.5 m²	7.2
derived: boat time	0.27 h per km of sea route	7.5

Appendix B. Scenario families (delta descriptions against the headline)

Each family changes the headline (or, for the earlier families, its own baseline) by the switch named. Origin labels and the literature behind each idea are taken from SCENARIO_PROVENANCE_v2.csv of the results package (column idea_group_origin; non-superseded rows; 486 scenario names). Quotations are not reproduced.

Family / idea group	Scenarios (examples)	Switch (config)	Idea origin
Baseline (Webster reference)	baseline	model.py without flags: default builders 50 + 25 per extra tower x U(0.9, 1.1), Webster pool, everyone cooperates	Authors' idea
Cooperation, crew formation	none, low, mixed, mostly, full	scenarios.list p_coop (cfg:119)	Authors' idea
Cooperation, help between sites	high	inter_site_help: true, 10 km radius	Authors' idea
Cooperation, patchwork	patchwork	site_coop_prob: 0.5	Authors' idea
TOST parameter-end tests	<level>_TOST_<param>_min/max	one baseline parameter at a range end (tost_configs/)	Authors' idea
Literature chaîne opératoire: Webster stages	webster	lit_scenarios.list	Literature: Webster 1991: 848-855; Webster 2001
S1a scree	S1a_scree	quarry: scree	Literature: Webster 1991: 848-849; Lilliu & Zucca 2005: 40-41
S1b jointed rock	S1b_joints_low, S1b_joints_high	quarry: joints_*	Literature: Webster 1991: 848; Webster 2015: 58; Serra 2021
S2a unworked	S2a_unworked	dress: none	Literature: Webster 1991: 850-851; Webster 2015: 59
S2b fine facing	S2b_fine	dress: fine	Literature: Contu 2002: 29-32; Paglietti & Mulas 2023
S3 rails	S3a_rails_tallow, S3b_rails_grease	mu 0.1 / 0.05	Literature: Webster 1991: 848-849; Webster 2015: 58
S3 ground	S3c_dry_ground, S3d_rough	mu 0.38 / 0.5	Literature: Webster 1991: 852; Gallin 1989; de Haan 2009 and Cotterell & Kamminga 1990 via Boswinkel 2021: 112-113
S3e oxen	S3e_oxen	oxen: true	Literature: Webster 1991: 849
S4a lever	S4a_lever	lift: lever	Literature: Cavanagh & Laxton 1987; Laner 1999

Family / idea group	Scenarios (examples)	Switch (config)	Idea origin
S4b ramp	S4b_ramp	lift: ramp	Literature: Laner 1999: 30; Webster 1991: 851; Webster 2015: 58
S4c internal stair	S4c_internal_stair	lift: stair	Literature: Laner 1992; 1999: 30, 42-47
S5a on site	S5a_onsite	onsite: true	Literature: Webster 1991: 848; Gallin 1989
S5b any building stone	S5b_any_buildstone	haul: buildstone	Literature: Lilliu & Zucca 2005: ~10-11; Tran et al. 2025
S5c far stone for complexes	S5c_complex_far	haul: complexfar	Literature: Stiglitz et al. 2012; Lo Schiavo et al. 2009
S6 density	S6_density	density: true	Literature (physical; densities and hardness assumed)
S7 Webster classes	S7_webster_classes	classes: true	Literature: Webster 1991
S8 learning	S8_learning	learning: true	Authors' idea
LOW / HIGH bounds	LOW, HIGH	cheapest / costliest option at every step	Authors' idea
REALISTIC	REALISTIC, REALISTIC_oxen	per-site mixed rules	Authors' idea
Alliance A0-A5	A0_alone, A1_ask, A2_trust, A3_trust_learning, A4_trust_track, A5_full, A5_full_freerider, A0_verify	alliance.list (cfg:282)	Authors' idea
Alliance sensitivity	27 SENS_R*_b*_t* cells	R, b, track cost on A5_full	Authors' idea
Social mechanisms (v13)	A5_full_freerider_V13rep, _V13pc, _V13reppc; MOST_REALISTIC_AUDITED_LIT_V13tie_P10, _V13all_P10	reputation, partner_choice, tie_imitation (config copies config/config_v13_*.yaml, queue config/queues/queue_v13_social_STAGED.tsv; v13 code; 7.15)	Literature form (Boyd & Richerson 2022; Apicella et al. 2012; Kasper & Borgerhoff Mulder 2015; Nowak & Sigmund 2005; Panchanathan & Boyd 2004); rules: Authors' idea
Social mechanisms, gentle reputation rule (v13g)	A5_full_freerider_V13repg, _V13repgpc; MOST_REALISTIC_AUDITED_LIT_V13allg_P10	as the v13 cells with reputation_rule: gentle (config copies config/config_v13g_*.yaml, queue config/queues/queue_v13_gentle.tsv; v13 code; 7.15)	Literature form (Fafchamps 1992; Boyd & Richerson 2022; Kasper & Borgerhoff Mulder 2015; Panchanathan & Boyd 2004); rule: Authors' idea
Pre-existing networks	P_<size>_<shape>, P_none, regional, stage-set, outcrop, D7 trust, D8 tie loss variants	pre_networks (code/pre_networks.py)	Authors' idea
Sea access S9	S9_sea_*, <scenario>_sea, sea grid with pre-networks	sea_kn / _sea suffix	Authors' idea
Phasing twins	<scenario>_P10, _P20	name suffix	Authors' idea
MOST_REALISTIC	MOST_REALISTIC_P10, _P20, _P100; _handover_*; _alone_*	alliance_extra (cfg:392)	Authors' idea
S10 staged complexes	MOST_REALISTIC_S10_*_P10	phase_order: {rule: staged}	Literature: Melis 2017: 40-41; Vanzetti et al. 2013: ~89 (unitary variant: Lilliu & Zucca 2005: 40-42; Contu 2002: 27-28)
S11 protonuraghi first	MOST_REALISTIC_S11_*_P10	phase_order: {rule: types_first}, type_cost	Literature: Ugas 2016: ~424-425; Webster 2015: 42; Moravetti 2017: 24
S12 food-limited crews	MOST_REALISTIC_S12_*_P10	food_cap	Literature: Dietrich et al. 2019; Dietler & Herbich 2001; Ialongo 2018; Webster 1991: 853
S13 antemurali and villages	MOST_REALISTIC_S13_walls*_P10, _S13_high_P10	extra_units	Literature: Moravetti 2000: 62-63, ~92; Webster 1996: ~119; Vanzetti et al. 2013: ~90
S14 maintenance	MOST_REALISTIC_S14_*_P10	maintenance	Literature: Foddai 2014: ~85-86; Lilliu 2005: ~186-187; Nieddu et al. 2016: ~25
C1 Stonehenge sledge crews	MOST_REALISTIC_C1_*_P10	haul_profile	Literature (comparative): Harris 2016; Webster 2001: 123
C2 Erasmus rates	MOST_REALISTIC_C2_*_P10	stage_rates, carry	Literature (comparative): Erasmus 1965: 286-7 via Webster 1991: 852
C3 Boswinkel rates	MOST_REALISTIC_C3_*_P10	stage_rates, haul_profile	Literature (comparative): Boswinkel 2021
M1 peer-polity emulation	MOST_REALISTIC_M1_*_P10	phase_order: {rule: emulation}	Literature (comparative): Renfrew 1986; Webster 2015

Family / idea group	Scenarios (examples)	Switch (config)	Idea origin
M2 mobile master masons	MOST_REALISTIC_M2_*_P10	masons	Literature (comparative): Araque Gonzalez 2021: 194-195; Bevan & Bloxam 2016
M3 wetted sand	MOST_REALISTIC_M3_*_P10	haul_profile.wet	Literature (comparative): Fall et al. 2014: 175502-3
M4 walked blocks	MOST_REALISTIC_M4_*_P10	haul_profile.walk	Literature (comparative): Lipo et al. 2013
v12 census S4 graded lift	MOST_REALISTIC_S4_graded_*_P10	lift_graded	Literature: Webster 1991: 851; Laner 1999: 30; Scintu 2003: 247; Webster 1991: 849; Webster 2001: 123
S4d spiral ramp	MOST_REALISTIC_S4d_spiral_*_P10	lift_ramp: {kind: spiral}	Literature: Stiglitz, Dies Cusi et al. 2015: 198-199
S4e Laner ramp	MOST_REALISTIC_S4e_laner_*_P10	lift_ramp: {kind: straight}	Literature: Laner 1999: 30
S2c vault dressing	MOST_REALISTIC_S2c_vault_*_P10	vault_dress	Literature: Webster 2015: 59; Laner 1999: 21-24
S15 foundations	MOST_REALISTIC_S15_found_*_P10	found	Literature: Webster 1991: 848, 851; 2001: 123
S16 mud mortar	MOST_REALISTIC_S16_mud_*_P10	mortar	Literature: Vanzetti et al. 2013: 89; Stiglitz et al. 2015: 198; Araque Gonzalez 2021: 193
S17 rubble fill	MOST_REALISTIC_S17_fill_*_P10	fill	Literature: Bortolussi et al. 2025: 12-13; Puddu 2014
S18 field clearance	MOST_REALISTIC_S18_clearance_*_P10	clearance	Literature: Araque Gonzalez 2021: 194-195
S19 mudbrick	MOST_REALISTIC_S19_mudbrick_*_P10	mudbrick	Literature: Chiri & Novelli 2022
S20 lintels	MOST_REALISTIC_S20_lintels_*_P10	lintel	Literature: Webster 2001: 21; Murru 2018: 229; Lilliu 1955: 158
S21 canton levy	MOST_REALISTIC_S21_levy_*_P10	levy	Literature: Moravetti et al. 2009: 361; Webster 1991: 853 (with masters: Lilliu 2009: 44-45)
S22 demography	MOST_REALISTIC_S22_demog_*	demog	Literature: Webster 1991: 853, Table 2; Ialongo 2018: 29
S23 bichrome masonry	MOST_REALISTIC_S23_bichrome_*_P10	bichrome	Literature: Foddai 2014; Stiglitz et al. 2015; Paglietti & Mulas 2023: 234
S13 rifascio	MOST_REALISTIC_S13_rifascio_*_P10	rifascio	Literature: Melis 2017: ~42; Lo Schiavo & Sanges 1994; Lilliu 1955: 158
M2 regional masons	MOST_REALISTIC_M2_regional_*_P10	mason_regions	Literature: Usai 2017
S12b feasts	MOST_REALISTIC_S12b_feast_*_P10	feast_attract	Literature (comparative): Dietrich et al. 2019; Araque Gonzalez 2021: 193-194
S24 carts	MOST_REALISTIC_S24_carts_P10	carts	Literature: Ugas 2016; Tagliagambe
S25 size by floors	MOST_REALISTIC_S25_size_P10	size_draw	Literature: Atzeni & Frau 2023: 326
Population	MOST_REALISTIC_POP_*_P10	builders_rule, population_ceiling	Authors' idea
Audited parameters	MOST_REALISTIC_AUD_*_P10, MOST_REALISTIC_AUDITED_LIT_P10	config copies / builders_rule	Literature: all audited values together (adult share from Webster 1991: 854, Table 2 n. 4)
Sensitivity: drag speed	MR_P10_v12_drag10	config copy, drag_speed_ms 1.0	Authors' idea
Sensitivity: pulling force	MR_P10_v11_pull125	config copy, 125 N	Authors' idea

Parameter alternatives. The full audited queue also runs single-parameter alternatives within published ranges: MOST_REALISTIC_ALT_*, MOST_REALISTIC_LOO_* and the config_full_audited_alt_* / _loo_* copies. These are not idea scenarios and have no row in SCENARIO_PROVENANCE_v2.csv; their sources are in LIT_ALT_COVERAGE.csv (results package).

Appendix C. Model development process

The model was developed by an AI research agent (the modelling agent) under the instructions of the principal investigator (PI).

- **The principal investigator** set the questions, specified the core mechanisms (asking for help outside one's own site, reciprocity-based trust, relays, the stone rule, sea access, phasing, the season length and the work-party rule), and approved or changed every assumption.
- **The modelling agent:**
  - searched and reviewed the archaeological and comparative literature;
  - proposed scenario families and parameter values, each labelled with its source or as an assumption;
  - wrote, ran and verified the code;
  - documented every change.
- **Code checks.**
  - The code was checked by a human: external code reviews by the PI, whose bug reports were each answered with a verdict, a fix, identity tests and a measured effect.
  - It was also checked by an independent AI agent, which reviewed fixes and re-ran the variance-based sensitivity analyses with its own kernels.
- **Verification** (`code/tests/`):
  - unit test of the Dijkstra search (`test_dijkstra.py`);
  - spot checks of routes (`verify_hauls.py`);
  - Webster calibration (`verify_webster.py`);
  - alliance checks: A0_verify equals webster, trust toy model, relay additivity, rank independence (`verify_alliance.py`);
  - sea, literature, scenario and pre-network checks;
  - reproducibility (`test_reproducibility.py`);
  - identity tests whenever code changed.
- **Audit trail.** Every parameter that acts on the headline was audited against the literature, with an evidence status and the action taken (PARAM_AUDIT.md, PARAM_AUDIT_ADDENDUM.md). A traceability matrix links each audited parameter to its configuration key, the code lines that read it and the queued scenarios that vary it (TRACEABILITY.md, PARAM_TRACE.csv); it also links each literature idea to its scenarios (IDEA_TRACE.csv). The audited values form the parameter set of the full audited run. The code of that run is frozen with checksums.

TRACE-oriented record. The elements are those of Grimm et al. (2014). This maps existing documents; no new TRACE document was written.

TRACE element	Where it is documented
Problem formulation	Section 1; paper draft (in preparation)
Model description	this ODD; `code/METHODS.md` (older in places, Appendix D)
Data evaluation	Section 6; PARAM_AUDIT.md Part 1; POPULATION.md (results package)
Conceptual model evaluation	PARAM_AUDIT.md Part 2 (behaviours, with sources and status)
Implementation verification	`code/tests/`; identity tests; code reviews
Model output verification	`code/tools/v12_checks.py` (patterns, 1.3); calibration against Webster (1991)
Model analysis	scenario families (Appendix B); parameter alternatives; earlier Sobol, Morris and TOST analyses
Model output corroboration	pattern checks b-i of 1.3 (to be re-run on the full audited outputs)

Appendix D. Discrepancies between the code and older documents

The code and configuration win in every case; this description follows them. "Docs" are `code/METHODS.md`, `param_audit/PARAM_AUDIT.md`, `param_audit/PARAM_AUDIT_ADDENDUM.md`, `param_audit/PARAM_TRACE.csv`, `param_audit/TRACEABILITY.md`, `AUDITED_LIT_SPEC.md` (results package), the paper draft (in preparation), and comments in the code and configuration.

#	Document says	Code / configuration does
D1	METHODS.md 9.1: Webster stages per tower "quarry 4,000, dress 10,400, walling 2,800 and lifting 2,800 ph"	`lit.stage_hours` with `lit.md_hours` (v11 day fix) gives quarry 2,500 ph (500 md x 5 h), dress 10,400, walling 2,800 and lifting 2,800. This applies to the `webster` stage set; the headline uses per-site rules (7.2)

#	Document says	Code / configuration does
D2	METHODS.md 9.1-9.2 and PARAM_AUDIT Part 2 rows 2b and 11b: walks at 1.4 m/s	`transport.return_walk_ms` = 1.39 (cfg:78), used in `trip` and `_join_task`
D3	METHODS.md 9.1: untried values start at 0 "so a community tries asking first"	`choose`: untried actions all count as 0 and ties are broken at random (`rng.integers`). Asking and not asking are equally likely on the first tower; an option tried once gets a negative value, so the untried one is preferred next
D4	METHODS.md 9.2: "7,159 sites have 5 faces, 2 have 1, and 6 have 0" (San Pietro); this implies 35,797 routes (7,159 x 5 + 2), a figure the document does not state	`cache/alliance/kbest.npz` used by the run: 7,165 sites with 5 faces, 2 with 1, none with 0; 35,827 routes
D5	PARAM_AUDIT Part 2 row 8a: "Builders = round(0.5 x residents)"; configuration comment on `mr_work_party` (around cfg:384): 0.5, "TWR1 5-40 -> 3-20"	`lit.builders_of`: max(1, floor(0.4 x r + 0.5)); single towers 2-16 builders (5.2)
D6	PARAM_AUDIT_ADDENDUM T-6, TRACEABILITY.md and the configuration comment at `unreachable_policy` (cfg:55): 6 San Pietro sites kept with a local quarry	In `sites_haul.csv`, `haul/sites_haul_mu0.38.csv` and `haul/sites_haul_mu0.1.csv`, `haul_flag` is "ok" for all 7,167 sites and no site has the stone "local_unlisted". With acid volcanic among the 7 permitted classes, no site is unreachable, so the policy acts on no site
D7	PARAM_AUDIT_ADDENDUM T-5: the snap radius "is large enough to move every mainland site off nodata"	`snap_status` is "ok" for all 7,167 sites in the tables used; no site is snapped (as the `nbcommon.snapped_cell` docstring says)
D8	AUDITED_LIT_SPEC.md ("Sea tables"): the cached sea table chooses land or sea at 0.6 m/s; "about 31 sites use the sea at 2 kn"	The run's own sea tables were rebuilt at 0.27 m/s and 1.39 m/s (`cache/sea/sea_meta.json`); 46 sites are in sea mode at 2 kn (`sites_haul_sea2kn.csv`)
D9	`tools/sea_routes.py` docstring: edge weight "cost/0.6 m/s" and "length/1.4 m/s"	`sea_params` reads `transport.drag_speed_ms` and `return_walk_ms` from the configuration (0.27, 1.39) for the route legs. People's walking speed in the sea tables is read from `scenarios.help_walk_ms` (cfg:117; `tools/sea_routes.py:472`), also 1.39, so there is no effect
D10	Configuration comments: geometry "15 m, truncated cone 12 m -> 8 m" (cfg:157); transport comments "Value unchanged" (cfg:75, cfg:77)	Values 14 m, 12 -> 6.8 m and 0.27 m/s are what the code reads (TRACEABILITY.md also notes these stale comments)
D11	PARAM_TRACE.csv and TRACEABILITY.md: T5 (`webster_day_hours`) "acts on MR_P10"	Partly true. Under the REALISTIC rules quarrying is always `joints_low` or `scree`, so the 5-h quarry day (and the transport day) never enter the headline. The dress and build entries (8 h) are read by `lit.stage_hours` (`lit.md_hours`, lines 86-88; `lit.stage_hours`, lines 95, 103) and do act: "acts on MR_P10" is right for dress and build, and wrong only for the quarry and transport hours (as in A.1, row T5)
D12	PARAM_AUDIT Q5: Webster's 500 / 1,300 / 700 man-days act on MR_P10	Only the dress (1,300) and build (700) man-days act; the quarry 500 md is replaced by the per-m³ quarry rule in every site
D13	Paper draft Table A3 and METHODS (S6): "density and hardness of the quarry stone"	`lit.mass_factor` and `lit.stage_hours` know only basalt, granite, limestone/marble and metamorphic. Acid volcanic (1,450 sites), andesite (517), schist (42), tuff (32) and trachyte (6), 2,047 sites in all, use Webster's 2,386 kg/m³ (mass factor 1) and hardness 1
D14	Paper draft 3.5 and Table A3: "routes up to 1 km use sledges on dry ground and longer routes use tallowed rails"; the density of "the actual stone"	The friction rule and the quarry stone are fixed per site from its mu 0.38 cheapest-outcrop route (`lit.mixed_haul`), whichever of the 5 faces is chosen. Face routes are the mu 0.3 paths re-costed at the site's friction (`kbest_mu038.py`), not re-optimised
D15	METHODS.md 9.1 and PARAM_AUDIT row 11a: relay hand-off "at f of the route's drag cost"	The hand-off move and the 300 flat-m minimum use the mu 0.3 cost (`Static.__init__`, lines 158-163; `start_tower`, line 1653). The legs are then costed at the site's own friction (mu 0.38 or 0.1)
D16	Paper draft (Tables A1-A3, Appendix A.1): drag 0.6 m/s, walk 1.4 m/s, workers = 0.5 x residents, jointed rock 11 ph/m³, and a different credit for the per-site building stone	Code and configuration: 0.27 m/s, 1.39 m/s, 0.4, 11.1 ph/m³; tower 14 m with top 6.8 m. The credit for the per-site building stone is Nurnet's data (Nurnet GeoPortal)
D17	`tools/v12_checks.py` (pattern checks, 1.3) and `tools/v12_summary.py`, `tools/family_summary.py`	They read the v11 run's outputs and use the v11 MOST_REALISTIC_P10 parts as the paired reference (paths written into the scripts). The V_duration note still says "work party = 0.5 x residents (3-20)". They have not yet been run against the full audited outputs

Appendix E. References

Protocol and methods (copied from the reference lists of Grimm et al. (2027) and the paper draft):

- Ayllón, D., Railsback, S.F., Gallagher, C., Augusiak, J., Baveco, H., Berger, U., et al., 2021. Keeping modelling notebooks with TRACE: good for you and good for environmental research and management support. Environ. Model. Software 136, 104932.
- Collier, N., Ozik, J., 2022. Distributed agent-based simulation with Repast4Py. In: Feng, B. et al. (Eds.), Proceedings of the 2022 Winter Simulation Conference. IEEE. https://doi.org/10.1109/WSC57314.2022.10015389

- Edmonds, B., le Page, C., Bithell, M., Chattoe-Brown, E., Grimm, V., Meyer, R., Montañola-Sales, C., Ormerod, P., Root, H., Squazzoni, F., 2019. Different modelling purposes. J. Artif. Soc. Soc. Simulat. 22 (3), 6. http://jasss.soc.surrey.ac.uk/22/3/6.
- Gilbert, N., 2020. Agent-Based Models, second ed. Quantitative Applications in the Social Sciences. SAGE, Thousand Oaks, CA. https://doi.org/10.4135/9781506355580
- Grimm, V., Augusiak, J., Focks, A., Frank, B.M., Gabsi, F., Johnston, A.S.A., Liu, C., Martin, B.T., Meli, M., Radchuk, V., Thorbek, P., Railsback, S.F., 2014. Towards better modelling and decision support: documenting model development, testing, and analysis using TRACE. Ecol. Model. 280, 129–139.
- Grimm, V., Railsback, S.F., Vincenot, C.E., Berger, U., Gallagher, C., DeAngelis, D.L., Edmonds, B., Ge, J., Giske, J., Groeneveld, J., Johnston, A.S.A., Milles, A., Nabe-Nielsen, J., Polhill, J.G., Radchuk, V., Rohwäder, M.-S., Stillman, R.A., Thiele, J.C., Ayllón, D., 2020. The ODD protocol for describing agent-based and other simulation models: a second update to improve clarity, replication, and structural realism. Journal of Artificial Societies and Social Simulation 23 (2), 7. https://doi.org/10.18564/jasss.4259
- Grimm, V., Rosello, C., Hamilton, S., Gallagher, C., Micheletti, T., Szangolies, L., Hengeveld, G.M., Pierce, M., Lemmen, C., Troost, C., Lee, A., Bell, A., Edmonds, B., Qin, C.-Z., Wimmler, M.-C., Appel, F., Ayllón, D., Banerjee, I., Barros, C., Berger, U., DeAngelis, D., Dong, C., Jagers, B., Jakeman, A., Kunkel, J., Müller, B., Njiru, R., Polhill, J.G., Radchuk, V., Razavi, S., Renton, M., Salinas, H., Zhu, L.-J., Evans, L.C., Seuru, S., Sun, X., Thibodeaux, C., Ullah, I.I., Vincenot, C., Wang, M., Zhang, F., Barton, M., Railsback, S., 2027. Twenty years of the ODD protocol: development, usage, extensions, and lessons learned about modelling standards development. Environmental Modelling and Software 207 (2027) 107203. https://doi.org/10.1016/j.envsoft.2026.107203 (open access, CC BY 4.0; available online 10 October 2026).
- Laatabi, A., Marilleau, N., Nguyen-Huu, T., Hbid, H., Ait Babram, M., 2018. ODD+ 2D: an ODD based protocol for mapping data to empirical ABMs. J. Artif. Soc. Soc. Simulat. 21 (2).
- Müller, B., Bohn, F., Dreßler, G., Groeneveld, J., Klassert, C., Martin, R., et al., 2013. Describing human decisions in agent-based models–ODD+ D, an extension of the ODD protocol. Environ. Model. Software 48, 37–48.
- Schmolke, A., Thorbek, P., DeAngelis, D.L., Grimm, V., 2010. Ecological models supporting environmental decision making: a strategy for the future. Trends Ecol. Evol. 25 (8), 479–486.
- Szangolies, L., Rohwäder, M.S., Ahmed, H., Jahanmiri, F., Wagner, A., Souto-Veiga, R., et al., 2024. Visual ODD: a standardised visualisation illustrating the narrative of agent-based models. J. Artif. Soc. Soc. Simulat. 27 (4).
- Wright, T.P., 1936. Factors affecting the cost of airplanes. Journal of the Aeronautical Sciences 3, 122–128. https://doi.org/10.2514/8.155

Social mechanisms (v13, 7.15):

- Apicella, C.L., Marlowe, F.W., Fowler, J.H., Christakis, N.A., 2012. Social networks and cooperation in hunter-gatherers. Nature 481, 497–501. https://doi.org/10.1038/nature10736
- Fafchamps, M., 1992. Solidarity networks in preindustrial societies: rational peasants with a moral economy. Economic Development and Cultural Change 41 (1), 147–174. (v13g gentle rule.)
- Boyd, R., Richerson, P.J., 2022. Large-scale cooperation in small-scale foraging societies. Evolutionary Anthropology 31, 175–198. https://doi.org/10.1002/evan.21944
- Kasper, C., Borgerhoff Mulder, M., 2015. Who helps and why? Cooperative networks in Mpimbwe. Current Anthropology 56 (5), 701–732. https://doi.org/10.1086/683024
- Nowak, M.A., Sigmund, K., 2005. Evolution of indirect reciprocity. Nature 437. (Cited for the concept of image scoring; not read at a page in this project.)
- Panchanathan, K., Boyd, R., 2004. Indirect reciprocity can stabilize cooperation without the second-order free rider problem. Nature 432. (Cited for the concept of standing; not read at a page in this project.)

Archaeological framing (copied from the paper draft):

- Anonymous (2026) [reference withheld for review]
- Araque González, R., 2021. Monuments of cooperating communities: Sardinian nuraghi and sanctuaries. In: Albertazzi, G., Muti, G., Saggio, A. (Eds.), Islands in Dialogue (ISLANDIA). Editoriale Artemide, Rome, pp. 193–200.
- ARPAS (Agenzia Regionale per la Protezione dell'Ambiente della Sardegna), 2019. Carta litologica della Sardegna, 1:25.000. CC BY.
- Boswinkel, Y., 2021. Labouring with Large Stones: A Study into the Investment and Impact of Construction Projects on Mycenaean Communities in Late Bronze Age Greece. Sidestone Press, Leiden.
- Calò, C.M., Melis, A., Vona, G., Piras, I.S., 2008. Sardinian population (Italy): a genetic review. International Journal of Modern Anthropology 1, 39–64. https://doi.org/10.4314/ijma.v1i1.60356 †
- Contu, E., 2002. Il Nuraghe Santu Antine. Carlo Delfino, Sassari.
- Hazell, L.C., Fitzpatrick, S.M., 2006. The maritime transport of prehistoric megaliths in Micronesia. Archaeology in Oceania 41, 12–24.

- Ialongo, N., 2018. Crisis and recovery: the cost of sustainable development in Nuragic Sardinia. European Journal of Archaeology 21, 18–38. https://doi.org/10.1017/eaa.2017.20
- Laner, F., 1999. Accabadora. Tecnologia delle costruzioni nuragiche. FrancoAngeli, Milan.
- Lilliu, G., Zucca, R., 2005. Su Nuraxi di Barumini. Carlo Delfino, Sassari.
- Richards, J., Whitby, M., 1997. The engineering of Stonehenge. Proceedings of the British Academy 92, 231–256.
- Turner, D.R., 2020. Grave Reminders: Comparing Mycenaean Tomb Building with Labour and Memory. Sidestone Press, Leiden.
- Webster, G.S., 1991. Monuments, mobilization and Nuragic organization. Antiquity 65, 840–856. https://doi.org/10.1017/S0003598X0008056X
- Webster, G.S., 2001. Duos Nuraghes: A Bronze Age Settlement in Sardinia. Vol. 1: The Interpretive Archaeology. BAR International Series 949. BAR Publishing, Oxford. https://doi.org/10.30861/9781841712413
- Webster, G.S., 2015. The Archaeology of Nuragic Sardinia. Equinox, Sheffield. https://doi.org/10.1558/isbn.9781781795309
- Webster, G.S., Webster, M.R., 1998. The Duos Nuraghes Project in Sardinia: 1985–1996 interim report. Journal of Field Archaeology 25, 183–201. https://doi.org/10.2307/530578
- Wunderlich, M., Müller, J., Behrens, A., 2024. Creating world views: work-expenditure calculations for Funnel Beaker megalithic graves and flint axe head depositions in Northern Germany. Open Archaeology 10, 20220357. https://doi.org/10.1515/opar-2022-0357 †

Other sources. The parameter and pattern citations in short form (for example de Haan 2009, Harris 2016, Tobler 1993, Sutton & Barto 2018, Sanna & Atzeni n.d., Lilliu 1955, Gallin 1989, Atzeni & Frau 2023) are those of the parameter audit. Their full references are in `param_audit/PARAM_AUDIT.md` and the paper (in preparation).

Data credits.

- Sites: Nurnet's data — a Nurnet merge and deduplication of various sources.
- Site building stone: Nurnet's data (Nurnet GeoPortal).
- Lithology: ARPAS 2019, Carta Litologica della Sardegna 1:25,000 (© RAS/ARPAS, CC BY 4.0).
- DEM: Copernicus DEM GLO-30 (© DLR e.V. 2010-2014 and © Airbus Defence and Space GmbH 2014-2018, provided under COPERNICUS by the European Union and ESA).
- Soils: Aru, Baldaccini & Vacca 1991.
- Rivers: Regione Autonoma della Sardegna.
- CRS: EPSG:25832.